DMarlon Studio

Manual de usuario

Manual de DMarlon Studio

Guía completa del IDE visual para crear aplicaciones de Python

Versión 1.0.114 · 2026-11-11

dmarlonstudio.com

Contenido

1 **Introducción e instalación**
Qué es DMarlon Studio · Requisitos · Instalación · Ejemplos incluidos · Dónde se guarda cada cosa

2 **Primeros pasos**
La ventana principal · Tema e idioma · Su primera ventana en cinco pasos

3 **El Diseñador**
Proyectos · La pantalla del Diseñador · Componentes · Mover, alinear y editar · Posición de la ventana · Eventos y código · Python generado y exportación · Ejecutar, detener y ver la salida · Conexiones de datos · Vista HTML: webs de verdad · El proyecto, su icono y el código

4 **Depurar**
Puntos de parada y depurar · Paso a paso · Variables, pila y evaluar · Historial, llamadas, rendimiento y objetos · Pruebas

5 **Electron + React**
Crear un proyecto de Electron · El código: componentes con propiedades · Ejecutar, depurar y exportar

6 **Extensiones de Chrome**
Crear una extensión · Lo propio de una extensión · Probar, cargar en Chrome y publicar

7 **Web con Next.js**
Una web con pantallas (como Flutter) · Una web con servidor · Una web de React (Vite) · Una web en HTML · Una web de Vue (Vite) · Una web con servidor en Go · Crear una página de Next.js

8 **C++ con FLTK**
Crear una aplicación de C++ · Compilar, ejecutar y exportar

9 **Pascal con Lazarus**
Crear una aplicación de Pascal · Compilar, ejecutar y exportar · Ejemplo: Odoo sin conexión y obras BIM

10 **Apps móviles con Flutter**
Crear una app de Flutter · La lógica y los enlaces · Escritorio · FalconDatabase y Firebase · El simulador y exportar

11 **Apps móviles con React Native**
Crear una app de React Native

12 **Aplicaciones de consola**
Crear un programa de consola · Ejemplos

13 **IA en el Diseñador**
Chat con Claude Code · Pedir a la IA · Presupuesto de IA · GitHub

14 **El Editor**
Abrir una carpeta · Explorador y pestañas · Buscar y reemplazar · Terminal · Control de código (git) · Claude implementa · Chat con la IA

15 **Bases de datos**
Conexiones · Tablas y consultas · FalconDatabase y Firebase

16 **Cliente de API**
Peticiones · Colecciones, variables e historial

17 **FTP**
Servidores y archivos

18 **Diagramas**
Diagramas de flujo y de base de datos

19 **Ajustes**
Conectar con Claude (MCP) · Inteligencia artificial · Idioma, apariencia y versión

20 **Atajos de teclado**
Diseñador · Editor

21 **Solución de problemas**
No encuentra Python o le falta tkinter · Falta Flask · La app no abre (sin firmar) · AttributeError: 'Form1' object has no attribute 'root' · No se encontró Claude Code

22 **Hoja de ruta**
Destinos · Proyectos más completos

Capítulo 1

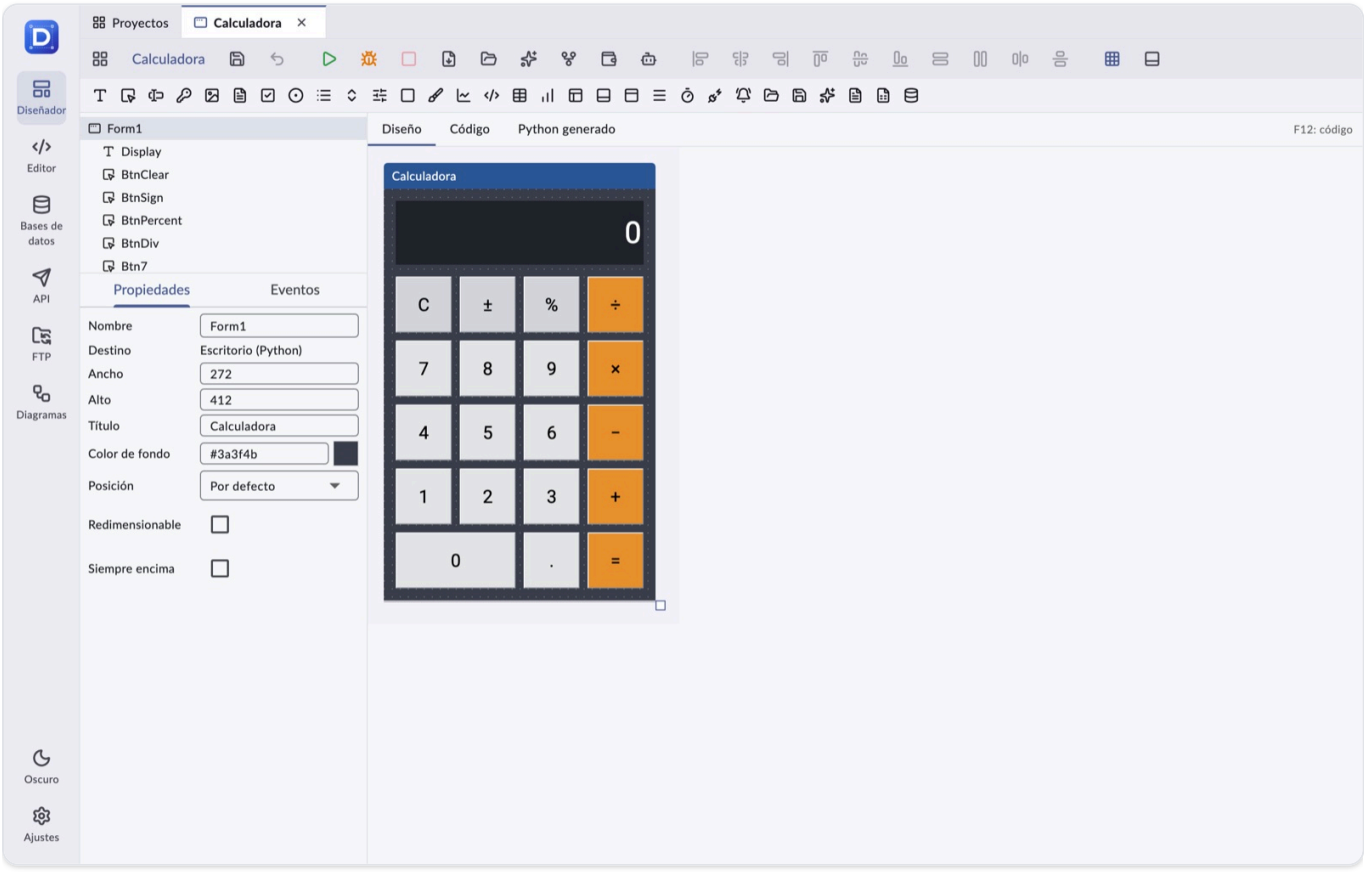
Introducción e instalación

Qué es DMarlon Studio, qué necesita y dónde guarda cada cosa.

Qué es DMarlon Studio

DMarlon Studio es un IDE visual de escritorio. Las ventanas se diseñan arrastrando componentes a un formulario, sus propiedades se ajustan en un inspector y el código de cada evento se escribe en Python. Con **F9** la ventana se ejecuta en su equipo.

Hoy genera dos tipos de aplicación con el mismo diseñador: ventanas de escritorio con **tkinter** y aplicaciones web con **Flask**. Alrededor del diseñador hay un editor de código al estilo de VS Code con git y terminal, un gestor de bases de datos, un cliente de API, un cliente FTP, un editor de diagramas y asistentes de inteligencia artificial (Claude por defecto).



El Diseñador: paleta arriba, árbol e inspector a la izquierda y el formulario en el centro.

Requisitos

Para…	Necesita
Usar el estudio	macOS 10.15 o posterior, o Windows 10/11 de 64 bits.
Ejecutar ventanas de escritorio	Python 3 con tkinter. En macOS: `brew install python-tk` o el instalador de python.org. En Windows: el instalador de python.org con la opción **tcl/tk and IDLE** marcada.
Ejecutar aplicaciones web	Flask en ese mismo Python: `python3 -m pip install flask` (en Windows, `py -3 -m pip install flask`).
Control de versiones en el Editor	git instalado en el equipo.
Chat con Claude Code en el Diseñador	Claude Code instalado y con la sesión iniciada (opcional).
Asistentes de IA	Una clave de API del proveedor elegido en **Ajustes** (opcional).

> El estudio busca Python por su cuenta: en el PATH, en las carpetas de Homebrew y en la instalación de python.org, y se queda con el que trae la versión de Tk más reciente. El resultado se ve en la lista de proyectos del Diseñador.

Instalación

Descargue el zip de su sistema desde la página de descargas de dmarlonstudio.com.

1. **macOS**: descomprima `DMarlonStudio-macOS.zip` y mueva **DMarlon Studio** a la carpeta Aplicaciones. La primera vez, abra la app con clic derecho → **Abrir**: no está firmada.
2. **Windows**: descomprima `DMarlonStudio-Windows.zip` completo donde quiera (no en Archivos de programa) y ejecute `DMarlonStudio.exe`. Si aparece «Windows protegió su PC», pulse **Más información** → **Ejecutar de todas formas**.

Ejemplos incluidos

Todos guardan sus datos en la carpeta del proyecto y funcionan sin configurar nada. Los marcados con (web) también existen como aplicación de Flask.

Ejemplo	Qué enseña
Calculadora (web)	Botones, eventos y estado
Conversor de unidades (web)	Listas desplegables y cálculo al escribir
Tareas (web)	Lista con altas, bajas y guardado
Cronómetro (web)	Temporizador, menú, barra de progreso y de estado
Ficha de cliente (web)	Control de páginas, listas y validación
Editor de notas (web)	Cinta de opciones y diálogos Abrir y Guardar
Consulta API (web)	Integración API en segundo plano y tabla de datos
Bloc de notas	Menú clásico, abrir, guardar, deshacer y barra de estado con línea y columna
Agenda	Contactos y citas en SQLite, búsqueda y páginas
Conversor de moneda	Tipos de cambio reales (BCE) con Integración API y reserva sin red
La serpiente	Juego en un lienzo con temporizador y récord guardado
Reproductor de música	Lista de reproducción, volumen y progreso (mejor con `pygame`)
Ventas mensuales (web)	Gráfico de líneas, área o barras y modo en vivo
Bolsa de valores (web)	Velas, media móvil y volumen con datos simulados en vivo
Ventanas y diálogos	Mensajes, preguntas, pedir datos, otra ventana que devuelve un resultado, un diálogo modal y ventanas no modales
Asistente de textos (web)	Claude para resumir, traducir, clasificar y extraer datos, y un chat con memoria
Informes (web)	Una tabla de ventas exportada a PDF y a Excel

Ejemplo	Qué enseña
Visor 3D	Figuras en 3D dibujadas en un lienzo: giro con el ratón, animación, luz y modo alambre
Correo	Recibir por IMAP y enviar por SMTP con adjuntos, sin congelar la ventana
CRM web, **Facturador web**	Aplicaciones web completas con SQLite
Blog web	Una web de verdad con la Vista HTML: portada, artículos, comentarios, boletín, cuentas con login y registro, y panel
Tienda web	Tienda de electrónica: ofertas, carrito, cuentas, pago simulado, «Mi cuenta» y panel de ventas y stock
Contactos de Odoo / ERPSqlite (web)	Conectarse a un Odoo por XML-RPC
Editor de PDF	Un paquete de pip (`pypdf`): girar, ordenar, unir, extraer y dividir páginas, sello de texto, contraseña y deshacer
Espacio en disco	Análisis en segundo plano sin congelar la ventana: árbol de carpetas, archivos más grandes y totales por tipo
Escáner de puertos	Red con sockets y muchas conexiones en paralelo, con progreso y exportación a CSV
Inversor de bolsa (demo)	Precios simulados en vivo, compra y venta con comisión, cartera guardada y gráfico
Punto de venta	Un TPV al estilo del de Odoo con imágenes: teclado de cantidad, descuento y precio, cobro, ticket y ventas en SQLite. Se conecta a Odoo con URL, usuario y contraseña para bajar productos y precios y enviar los pedidos a Ventas
Renombrar archivos	Reglas con expresiones regulares y numeración, vista previa con conflictos y deshacer
Odoo sin conexión (Pascal)	Una app nativa de Lazarus para Odoo 8 a 20 por JSON-RPC: panel con gráficos, clientes, proveedores, productos, ventas y compras de principio a fin (presupuesto o solicitud, confirmación, factura y pago en banco o efectivo) y un punto de venta que crea pedido, entrega y factura, con y sin conexión y sin duplicados. Con BIM 2.0 (base_bim_2) en Odoo, también obras y presupuestos por conceptos (capítulos, partidas y recursos de mano de obra, equipo, material y subcontrata) que se hacen sin conexión y se suben a Odoo

Ejemplo	Qué enseña
Seguimiento de trenes	Datos públicos en directo (GTFS-Realtime de Renfe) en segundo plano, alertas de proximidad, malla de circulación y mapa dibujados en lienzos, y una simulación sin conexión
Reproductor MP3	Una interfaz propia dibujada en lienzos (reloj digital, espectro), etiquetas ID3, listas M3U y sonido con `pygame`
Explorador de archivos (C++)	Una app nativa en C++ con FLTK: componentes propios dibujados en código (árbol que se carga al abrirlo, vistas de iconos, lista y detalles, pestañas, migas en la dirección), marcadores guardados, copiar y pegar con el portapapeles de archivos del sistema (entre ventanas y con el Finder o el Explorador), Papelera, propiedades calculadas en segundo plano y tema claro u oscuro
Editor de imágenes (C++)	Un editor de imágenes nativo en C++ con FLTK: el Timer del diseño anima la selección; la barra de herramientas, el lienzo con zoom (nítido en pantallas Retina), los paneles Color, Capas e Historial y los diálogos con vista previa se dibujan en código; capas con opacidad y ocho modos de fusión, un historial que comparte las capas que no cambian, ajustes y filtros, recortar, cambiar el tamaño con remuestreo bilineal o bicúbico, girar cualquier ángulo y codificadores JPEG y BMP propios (abre lo que lee FLTK y guarda las capas en .dmi)
Buscador de vuelos (C++)	Una app nativa en C++ con FLTK que vigila rutas: el Timer del diseño lanza un análisis cada 30 minutos; las consultas a la API de Travelpayouts (Aviasales) van con el programa curl en otro hilo y el JSON se lee a mano; el mejor precio de cada aerolínea con su enlace para comprar, el precio de cada día del mes y la evolución del mejor precio se dibujan en código, con avisos del sistema cuando baja; sin token, datos de ejemplo creíbles
Inspector del equipo (C++)	Una app nativa en C++ con FLTK que analiza el ordenador: sysctl y system_profiler en JSON (macOS) o la API de Win32 y PowerShell con Get-CimInstance (Windows) en otro hilo; el Timer del diseño mide cada segundo el procesador y cada núcleo, la memoria, la red, los discos y la batería; las tarjetas, las gráficas y las tablas se dibujan en código, y el informe de varias páginas sale en PDF con Fl_PDF_File_Surface o en la impresora con Fl_Printer, con los números de serie ocultos salvo que se pidan
Carrera de motos (C++)	Un juego nativo en C++ con FLTK: el Timer del diseño mueve la carrera 60 veces por segundo; la pista en pseudo-3D (segmentos proyectados con curvas y cuestas), las motos que se inclinan, el paisaje y el cuadro de mandos se dibujan en código; siete rivales que frenan, trazan y adelantan, vueltas cronometradas, clasificación con diferencias, minimapa y récord guardado
Bajo la Sombra del Malecón (Pascal)	Una aventura de investigación en Object Pascal con Lazarus: una pantalla VGA de 360×220 y 256 colores dibujada píxel a píxel en un TPaintBox (escenas con tramado, retratos, iconos y letras sacadas de la fuente del sistema sin suavizar), el Timer del diseño para el texto que se escribe y lo que se mueve, cuatro actos con pistas y objetos, y tres finales

Ejemplo	Qué enseña
Docker en la barra de menús (Pascal)	Una app de la barra de menús en Object Pascal con Lazarus: un TTrayIcon con su menú (en macOS, una plantilla que se pinta del color de la barra y sin icono en el Dock mientras no se abre la ventana), «docker ps» en un hilo cada tres segundos con el Timer del diseño, las direcciones sacadas de los puertos publicados, parar, reiniciar e iniciar contenedores, y un panel dibujado en código
Lanzador de apps (Pascal)	Una app de la barra de menús en Object Pascal con Lazarus que lanza las aplicaciones de su propia carpeta: el menú lleva el icono de cada app (NSWorkspace, en 16 puntos), favoritas, recientes y un panel con buscador (el Entry del diseño con su evento Intro) que no distingue acentos
Monitor de sitios web (C++)	Una app nativa en C++ con FLTK: los sitios se añaden con los campos del diseño y se comprueban con curl en otros hilos cada 1 a 30 minutos (Combobox del diseño), con su código, su tiempo de respuesta, la disponibilidad y un historial en barras dibujado en código; en macOS vive también en la barra de menús (un NSStatusItem hecho desde C++ con objc_msgSend) y avisa cuando un sitio se cae o vuelve; además, un escáner de los 33 puertos habituales de cada servidor (sockets no bloqueantes) que marca los que no conviene abrir a internet
Relojes del mundo (C++)	La hora de varios países en la barra de menús en C++ con FLTK: las reglas del horario de verano (Unión Europea, Estados Unidos, Cuba, Chile, Australia y Nueva Zelanda) van en el código y se comprueban contra la base de zonas del sistema; esferas dibujadas en código, oscuras de noche, y la barra de menús hecha con el runtime de Objective-C
Grabador de pantalla (C++)	Graba vídeos para YouTube en C++ con FLTK: la pantalla entera o una zona de 16:9 o vertical, con micrófono, clics y cursor, cuenta atrás y límite; graba con screencapture en macOS (o ffmpeg en Windows y Linux), esconde la ventana y lleva el tiempo en la barra de menús; la duración y el tamaño se leen del propio vídeo y las miniaturas son de QuickLook

Ejemplo	Qué enseña
MyWhatsapp (C++)	Lee los chats de WhatsApp Desktop (su base de datos SQLite, solo lectura, con el SQLite del sistema cargado con dlopen) o un chat exportado, y propone tres respuestas con Claude (claude-opus-5-5, esfuerzo bajo) o ChatGPT por su API HTTP con curl; las claves van en el Llavero; con las fotos de perfil (Media/Profile), las fotos y documentos recibidos, quién escribe en los grupos (con el nombre de la agenda) y tema claro, oscuro o como el sistema; envía con «Enviar ➤» o Enter, también adjuntos, por la API oficial de WhatsApp Business (Graph API) o vinculando WhatsApp con un QR mediante wabridge, un puente en Go con whatsmeow que va junto a la app; tareas desde un chat («+ Tarea» o clic derecho sobre un mensaje), en una vista propia y guardadas en tareas.jsonl; seleccionar y copiar texto, buscar en el chat (Cmd+F) y subir por el historial; las tareas pendientes llevan una bolita verde, naranja o roja según su edad; los mensajes repetidos de Desktop se juntan por su id; chatear en otro idioma con «🌐 Traducir» (traducción de lo recibido y de lo enviado, por contacto), la bandera y la hora local de cada contacto, y los stickers (WebP, convertidos con sips); filtros en la lista (con tareas, por responder en 3 días, sin respuesta) y los chats archivados aparte; las fotos se abren en grande en un visor, con «⬇ Descargar»; modos ocupado (los que quiera, cada uno con su mensaje) que contestan solos una vez a quien escribe en privado; en la barra de menús de macOS, con cuántos chats esperan respuesta y un menú con ellos; al cerrar la ventana sigue ahí; los modos ocupado pueden contestar con IA según la conversación; «✨ Mejorar» y «Aa Corregir» (la IA mejora o corrige lo que escribe); un diseñador de estados al estilo Canva (plantillas, textos con IA, foto) que los guarda como imagen y los publica en WhatsApp
Dominó doble 9	Un juego de mesa en tkinter: la mesa, las fichas y la cadena que gira al llegar al borde se dibujan en un Lienzo con sus eventos de clic y de ratón; el Timer del diseño hace jugar a la computadora (que recuerda a qué números pasó cada uno); dominó, tranque y partida a 100 tantos

Veintiocho de estos ejemplos se pueden descargar ya compilados para macOS y Windows en la página **Demos** de dmarlonstudio.com, y tres apps móviles y siete webs se prueban allí mismo en el navegador (las móviles, también como APK de Android). Cada demo de escritorio es una sola aplicación en español y en inglés: el idioma se cambia en su menú **Idioma** (Language). Cada demo tiene su página con cómo funciona en tres pasos: son la mejor forma de ver lo que se puede hacer con el estudio antes de instalarlo.

Dónde se guarda cada cosa

Qué	Dónde
Proyectos del Diseñador	Una carpeta por proyecto en `~/Documents/DMarlon Studio/Proyectos` (se puede cambiar en el Diseñador). Se pueden versionar con git.

Qué	Dónde
Ajustes, conexiones, peticiones de API, historial y diagramas	`studio.sqlite`, en la carpeta de datos de la aplicación: `~/Library/Application Support/com.dmarlonstudio.studio` en macOS y `%APPDATA%\DMarlon Studio\DMarlon Studio` en Windows.
Proyectos borrados	La carpeta `.papelera` dentro del área de trabajo: nada se borra del todo.

Cada carpeta de proyecto contiene `project.dmsproj` (el diseño en JSON), el código de la ventana (`Form1.py`, con el nombre del formulario), un `.gitignore` y, al ejecutar, `build/` con el Python generado.

Capítulo 2

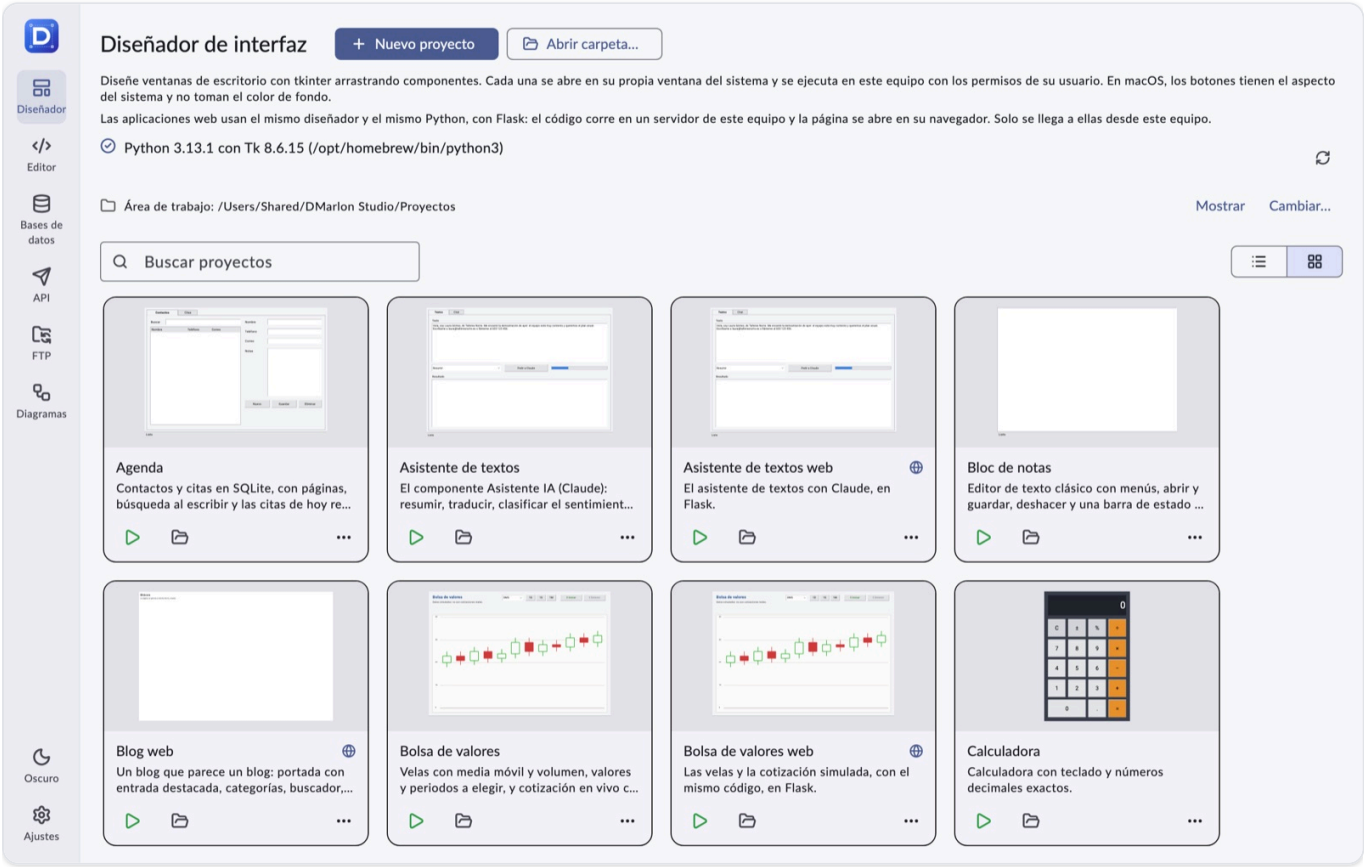
Primeros pasos

La ventana principal, los módulos, el tema y el idioma.

La ventana principal

A la izquierda está la barra de módulos: **Diseñador**, **Editor**, **Bases de datos**, **API**, **FTP**, **Diagramas** y, abajo, el botón de modo **Oscuro**/**Claro** y **Ajustes**.

Cada módulo se crea la primera vez que se abre y conserva su estado: puede pasar del Diseñador al Editor y volver sin perder lo que no haya guardado.



La lista de proyectos, con los ejemplos que trae el estudio.

Tema e idioma

- El botón **Oscuro**/**Claro** de la barra cambia el tema al momento. En **Ajustes → Apariencia** también puede elegir **Sistema** para seguir al del equipo.
- **Ajustes → Idioma** cambia entre **Español** e **English** sin cerrar nada. La primera vez se usa el idioma del sistema.

Su primera ventana en cinco pasos

1. En **Diseñador**, pulse **Nuevo proyecto**, escriba un nombre y elija **Escritorio (Python)**.
2. Arrastre una **Etiqueta**, un **Campo de texto** y un **Botón** desde la paleta al formulario.
3. Seleccione el botón y cambie su **Texto** en el inspector, por ejemplo a «Saludar».
4. Haga doble clic en el botón: se crea el método `Button1_click` y se abre el código. Escriba `self.Label1.config(text="Hola, " + self.Entry1.get())`.
5. Pulse **F9**. La ventana se abre en su propia ventana del sistema.

Capítulo 3

El Diseñador

Proyectos, componentes, inspector, eventos, código generado y ejecución.

Proyectos

La lista de proyectos muestra los del área de trabajo y los abiertos desde otras carpetas. Arriba se ve el Python encontrado (versión de Python y de Tk) y el botón para **Volver a buscar Python**.

Se ve como una **lista** con columnas (**Nombre**, **Tipo**, **Componentes**, **Modificado** y **Carpeta**) que se ordenan con un clic en su título, o en **Tarjetas**. Tiene **Buscar proyectos** y va por páginas (**Por página**: 10, 15, 25 o 50). Doble clic abre un proyecto. El botón derecho sobre un proyecto ofrece **Abrir**, **Ejecutar**, **Mostrar la carpeta**, **GitHub…**, **Presupuesto de IA**, **Duplicar** y **Eliminar**; sobre el fondo, **Agregar nuevo** › **Proyecto de escritorio (Python)**, **Proyecto web (Flask)** o **Abrir carpeta…**.

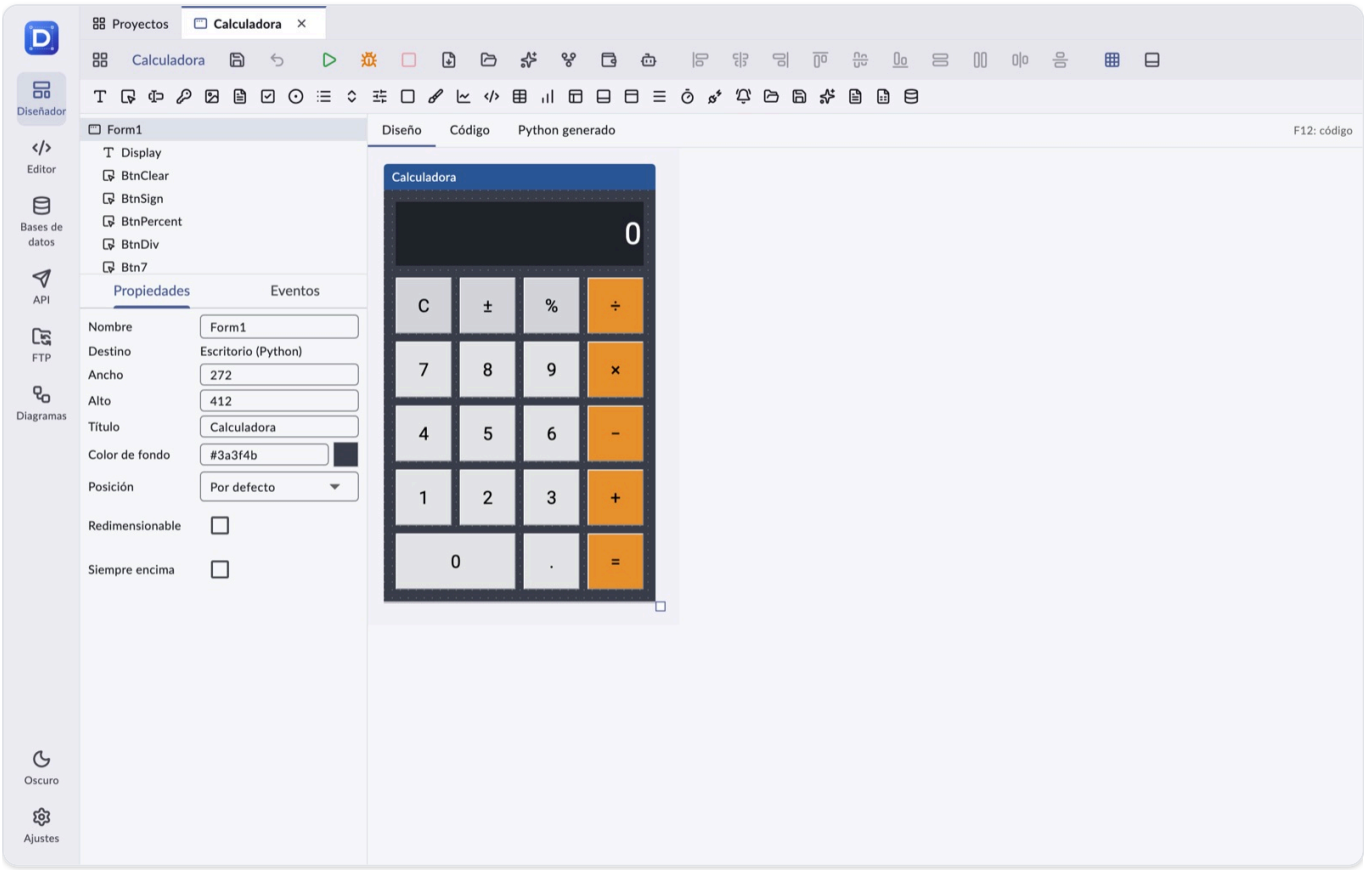
- **Nuevo proyecto**: pide el nombre y el destino, **Escritorio (Python)** o **Web (Flask)**. El destino no se puede cambiar después.
- **Abrir carpeta…**: abre un proyecto que esté en cualquier otra carpeta; queda en la lista como reciente.
- **Área de trabajo**: **Mostrar** la abre en el Finder o el Explorador y **Cambiar…** elige otra carpeta.
- En cada tarjeta: **Ejecutar**, **Mostrar la carpeta** y, en **Más acciones**, **GitHub…**, **Presupuesto de IA**, **Duplicar** y **Eliminar** (lleva el proyecto a la papelera del área de trabajo).
- **Pestañas**: cada proyecto abierto tiene su pestaña arriba, con un punto si hay cambios sin guardar. Puede tener varios abiertos a la vez; la pestaña **Proyectos** vuelve a la lista sin cerrar nada. Se cierran con la ✕ o con clic central, y preguntan si guardar.

> ⓘ El estudio copia en el área de trabajo los proyectos de ejemplo (los nuevos aparecen también en áreas de trabajo ya creadas). Vea la lista completa en **Ejemplos incluidos**.

La pantalla del Diseñador

- **Barra de herramientas**: volver a los proyectos, nombre de la aplicación (clic para renombrarla), **Guardar**, **Deshacer**, **Ejecutar (F9)** en verde, **Detener** en rojo, **Abrir en el navegador** (web), **Mostrar la carpeta del proyecto**, **Exportar el archivo .py**, **GitHub**, **Presupuesto de IA**, **Pedir a la IA**, **Chat con Claude Code**, alineación y centrado, **Ajustar a la rejilla** y **Ver la salida**.
- **Consola de salida**: abajo; se agranda o se reduce arrastrando su borde superior, se minimiza con la flecha y **Copiar la salida** copia todos los mensajes.
- **Paleta**: una barra de iconos; al pasar el ratón sale el nombre del componente. Se arrastra al formulario o se añade con doble clic.

- **Árbol**: la ventana y sus componentes, con los que están dentro de un marco o de una página sangrados.
- **Inspector**: **Propiedades** y **Eventos** del componente seleccionado o, si no hay ninguno, de la ventana.
- **Centro**: pestañas **Diseño**, **Código** y **Python generado**. **F12** pasa del diseño al código y vuelta.



La calculadora de ejemplo en el Diseñador.

Componentes

Todos los componentes tienen **Nombre**, posición y tamaño. Casi todos admiten **Color de fondo**, **Color del texto** y fuente (**Fuente**, **Tamaño de fuente**, **Negrita**, **Cursiva**), y los eventos de ratón (**Clic**, **Doble clic**, **Clic derecho**, **Entrar el ratón**, **Salir el ratón**). Los de escritura añaden los de teclado (**Recibir el foco**, **Perder el foco**, **Tecla pulsada**, **Tecla soltada**, **Tecla Intro**).

Componente	Propiedades propias	Eventos principales
Etiqueta (`Label`)	Texto, Alineación, Justificación, Relieve, Ajuste de línea	Ratón
Botón (`Button`)	Texto, Estado, Relieve	**Al pulsar** (`Button1_click`)
Campo de texto (`Entry`)	Texto inicial, Justificación, Ocultar con (p. ej. `*` para contraseñas), Estado, Tabla enlazada y Campo enlazado	Teclado y ratón

Componente	Propiedades propias	Eventos principales
Contraseña (`Password`)	Texto de ayuda, Botón para ver la contraseña, Longitud máxima, Estado	Teclado y ratón
Imagen (`Image`)	Archivo (en la carpeta `images` del proyecto), Ajuste (tamaño original, entera sin deformar, rellenar recortando, estirar), Borde	Ratón
Texto largo (`Text`)	Texto inicial (varias líneas), Ajuste de línea, Estado	Teclado y ratón
Casilla (`Checkbutton`)	Texto, Estado, Marcada	**Al pulsar**
Opción (`Radiobutton`)	Texto, Estado, Grupo, Valor, Marcada	**Al pulsar**
Lista (`Listbox`)	Elementos (uno por línea), Selección (browse, single, multiple, extended)	**Al seleccionar**
Lista desplegable (`Combobox`)	Elementos (uno por línea), Texto inicial, Justificación, Estado (normal, readonly, disabled)	**Al seleccionar**
Deslizador (`Scale`)	Título, Desde, Hasta, Paso, Valor, Orientación, Mostrar el valor	**Al cambiar** (recibe el valor)
Marco (`Frame`)	Relieve, Borde. Contiene otros componentes	Ratón
Lienzo (`Canvas`)	Color de fondo. Para dibujar desde el código	Ratón, **Arrastrar**, **Soltar el botón**, **Mover el ratón**
Tabla de datos (`Treeview`)	Fuente de datos, Consulta (SQL; en MongoDB, JSON), Columnas (campo|título|ancho), Límite, Orden, Cargar al abrir	**Al seleccionar**
Barra de progreso (`Progressbar`)	Valor, Máximo, Modo (determinate o indeterminate), Orientación	Ratón
Control de páginas (`PageControl`)	Páginas (una por línea), Página activa. Cada página contiene sus componentes	**Al cambiar de página**
Barra de estado (`StatusBar`)	Texto. Ocupa siempre el borde inferior de la ventana	Ratón

Componente	Propiedades propias	Eventos principales
Cinta de opciones (`Ribbon`)	Pestañas, grupos y botones (con sangría; «Botón | icono»), Pestaña activa	**Al pulsar un botón**, **Al cambiar de pestaña**
Menú (`Menu`)	Opciones (una por línea; sangría = submenú; `-` = separador). No visual: se ve arriba de la ventana	**Al elegir una opción** (recibe la opción)
Temporizador (`Timer`)	Intervalo (ms), Activo. No visual	**Cada intervalo**
Integración API (`RestClient`)	URL base, Autenticación (ninguna, token Bearer, básica o clave en una cabecera; el secreto, mejor en una variable de entorno), Cabeceras, Tiempo máximo. No visual	**Al terminar una llamada**, **Si falla una llamada**
Icono de bandeja (`TrayIcon`)	Texto al pasar el ratón, Imagen, Menú (una opción por línea). No visual	**Al hacer clic en el icono**, **Al elegir una opción del menú**
Diálogo Abrir (`OpenDialog`)	Título, Tipos («Texto|*.txt»), Extensión por defecto, Carpeta inicial, Elegir varios archivos. No visual	**Al elegir un archivo**
Diálogo Guardar (`SaveDialog`)	Título, Tipos, Extensión por defecto, Carpeta inicial, Nombre del archivo, Preguntar antes de sobrescribir. No visual	**Al elegir un archivo**
Asistente IA (`AiAssistant`)	Modelo de Claude, Instrucciones, Esfuerzo, Longitud máxima. No visual. Desde el código: `ask`, `chat` (con memoria; `reset` empieza otra), `summarize`, `translate`, `classify` (una etiqueta de una lista), `extract` (datos en un diccionario) y sus variantes `*_async`, que no congelan la ventana. Usa el SDK oficial de Anthropic (`pip install anthropic`); al ejecutar desde el estudio recibe la clave de **Ajustes** por la variable `ANTHROPIC_API_KEY`	**Al llegar la respuesta**, **Si falla la petición**
Informe PDF (`PdfReport`)	Título del documento, Nombre del archivo, Orientación, Autor, Números de página. No visual. Desde el código: `title`, `heading`, `text`, `table` (con totales y cabecera repetida en cada página), `image` (JPEG), `line`, `page_break`; luego `save`, `save_dialog`, `open` o `download` (web). Sin bibliotecas que instalar	—
Informe Excel (`ExcelReport`)	Nombre del archivo, Primera hoja. No visual. Desde el código: `sheet`, `header`, `write_row`, `write_rows` (texto, números, fechas y casillas con su tipo; cabecera fija con filtro); luego `save`, `save_dialog`, `open` o `download` (web)	—

Componente	Propiedades propias	Eventos principales
Gráfico (`Chart`)	Tipo (líneas, área, barras o velas), Título, Títulos de los ejes, Leyenda, Rejilla, Decimales, Prefijo y Sufijo de los valores, Puntos como máximo. Desde el código: `set_series` , `add_point` , `set_candles` , `add_candle` , `clear`	**Al hacer clic en un punto**
Conexión de datos (`Connection`)	Conexión (una de las guardadas en **Bases de datos**). No visual	—

Menú, Barra de progreso, Barra de estado, Temporizador y Control de páginas llegan con la versión 1.0.5; Contraseña, Imagen, Cinta de opciones, Integración API, Icono de bandeja y los diálogos Abrir y Guardar, con la 1.0.6; el Gráfico, con la 1.0.8. En una aplicación web, los componentes sin equivalente lo indican en el inspector.

Mover, alinear y editar

- Clic selecciona; arrastrar mueve; los tiradores cambian el tamaño. Con **Ajustar a la rejilla** activado todo encaja en la rejilla.
- Las flechas mueven la selección un paso de rejilla; con Alt (Opción), un píxel; con Mayús, cambian el tamaño.
- Con dos o más componentes seleccionados se activan **Alinear a la izquierda**, **Centrar en horizontal**, **Alinear a la derecha**, **Alinear arriba**, **Centrar en vertical**, **Alinear abajo**, **Igualar el ancho** e **Igualar el alto** (respecto al primero seleccionado).
- Con uno o más seleccionados, **Centrar en la ventana (horizontal)** y **Centrar en la ventana (vertical)** los centran como un bloque en la ventana o en su marco.
- El botón derecho ofrece **Cortar**, **Copiar**, **Pegar**, **Eliminar**, **Traer al frente** y **Enviar al fondo**.
- **Copiar entre proyectos**: lo copiado va al portapapeles del sistema, con sus hijos y **el código de sus eventos**. Se pega en otro proyecto (o tras reiniciar el estudio): los componentes toman nombres libres y sus métodos se renombran con ellos, sin pisar ninguno que ya exista. Los métodos auxiliares que llamen sus eventos se copian a mano.
- **Deshacer** (⌘Z / Ctrl+Z) vuelve atrás cada cambio del diseño. Si el código se cambió a mano después de un paso, ese paso restaura el diseño pero no el código, y se avisa.

Posición de la ventana

La propiedad **Posición** de la ventana decide dónde se abre: **Centrada en la pantalla** (la de los proyectos nuevos), **Por defecto** (la decide el sistema) o **Manual**, con **Izquierda** y **Arriba**.

Eventos y código

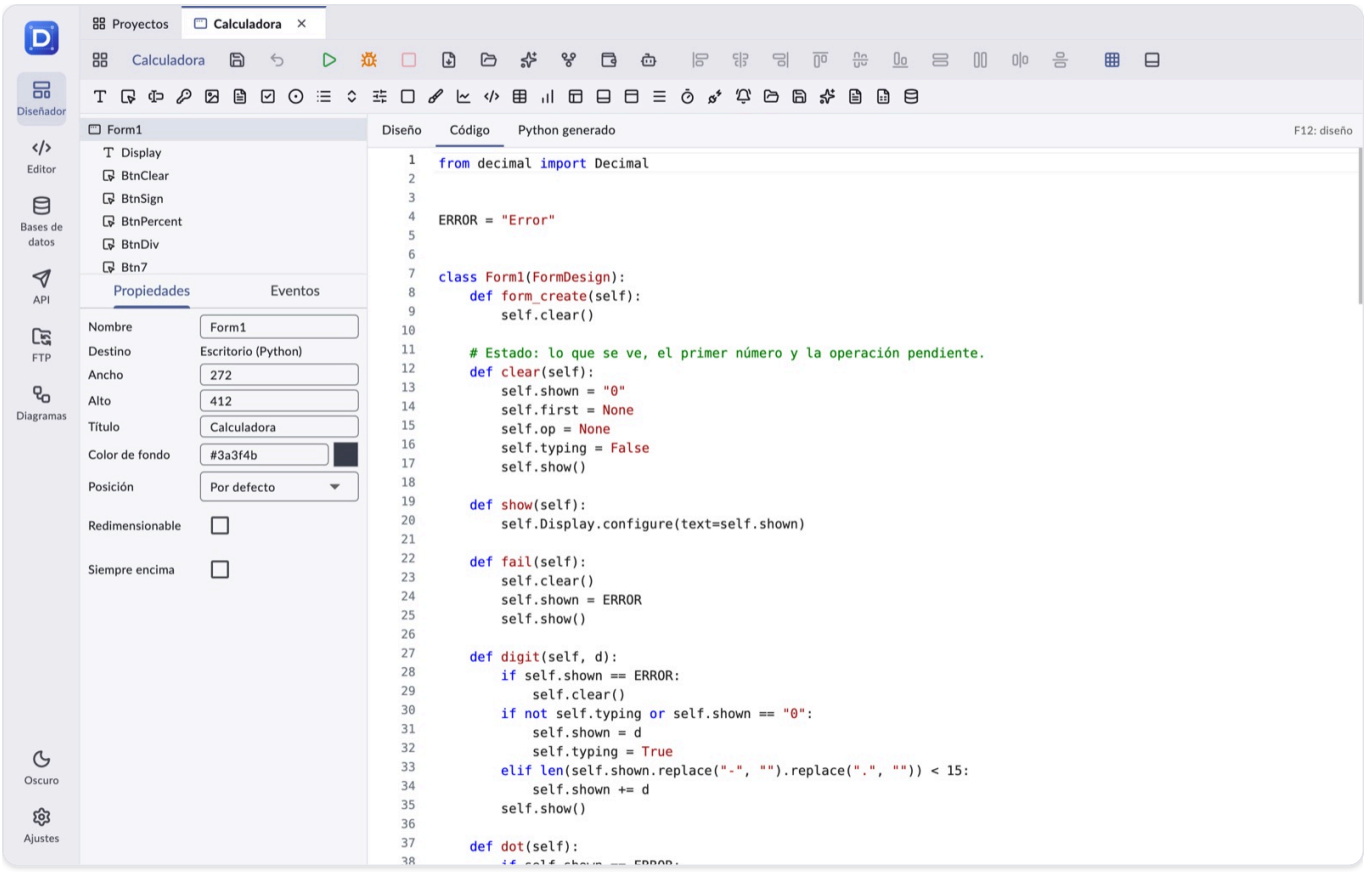
El código de la ventana es una clase de Python con el nombre del formulario que hereda de `FormDesign`, la clase que genera el estudio. `self.root` es la ventana y cada componente es `self.<Nombre>`.

- Doble clic en un componente abre su evento principal (`Button1_click`, `Listbox1_select`…) y lo crea si no existe.
- En la pestaña **Eventos** del inspector se escribe o se elige el método de cada evento; **Ir al código** salta a él.
- Renombrar un componente renombra sus métodos de serie y las referencias del código.
- La ventana tiene sus eventos: **Al abrir** (`form_create`), **Al cerrar**, **Tecla pulsada** y **Al cambiar de tamaño**, y sus propiedades **Título**, **Color de fondo**, **Redimensionable** y **Siempre encima**.

```
class Form1(FormDesign):
    def form_create(self):
        self.total = 0
        self.StatusBar1.config(text="Listo")

    def Button1_click(self):
        self.total += 1
        self.Label1.config(text=f"Pulsado {self.total} veces")
```

⚠ Ponga lo que deba ocurrir al arrancar en el evento **Al abrir** (`form_create`). Si define su propio `__init__`, llame primero a `super().__init__()`.

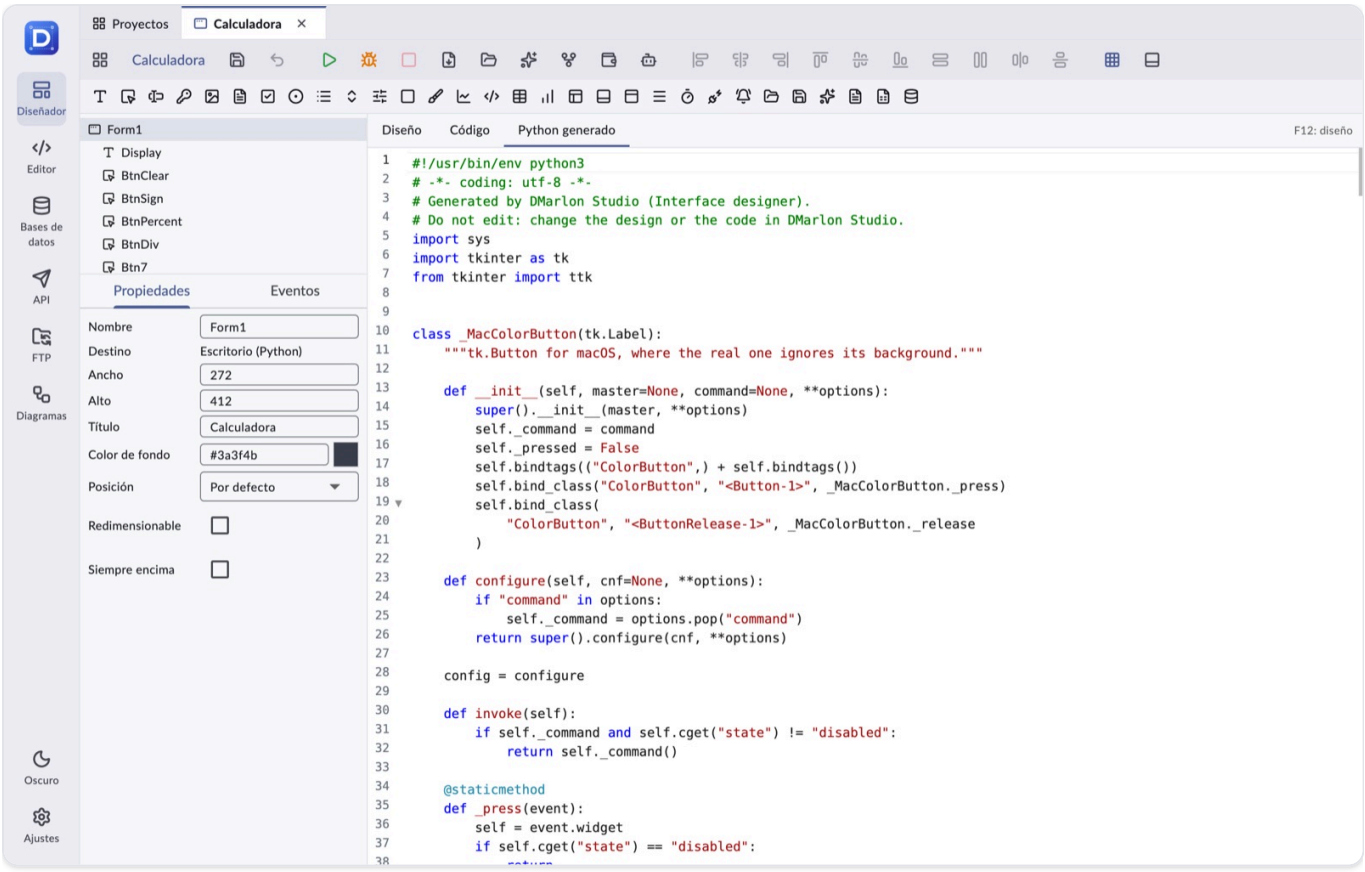


La pestaña Código: la clase de la ventana con sus manejadores.

Python generado y exportación

La pestaña **Python generado** muestra, solo para leer, el archivo completo que se ejecuta: el diseño convertido en código más su clase. Al ejecutar se escribe en `build/<proyecto>.py`, dentro de la carpeta del proyecto.

Exportar el archivo .py guarda ese archivo donde quiera. Funciona con cualquier Python 3 con tkinter (o con Flask, si es web) sin el estudio.



El Python generado: no se edita, se rehace desde el diseño y el código.

Ejecutar, detener y ver la salida

- **Ejecutar (F9)** guarda el proyecto y lanza la ventana en su propia ventana del sistema, con los permisos de su usuario.
- **Detener** cierra la ventana o el servidor en marcha.
- **Ver la salida** abre el panel con lo que la aplicación escribe (`print`, errores de Python) y el Python usado.
- En una aplicación web, el estudio arranca el servidor de Flask en este equipo y abre el navegador; **Abrir en el navegador** vuelve a abrirla. Solo se llega a ella desde este equipo.
- Si falta un paquete de Python (Flask, el SDK de Anthropic, un controlador de base de datos o pystray), el estudio lo dice antes de ejecutar y ofrece **Instalar**: lo instala con el pip del mismo Python que ejecuta el proyecto y después sigue. También puede **Copiar la orden** para hacerlo a mano.

Conexiones de datos

1. Guarde la base de datos en el módulo **Bases de datos** (SQLite, PostgreSQL, MySQL/MariaDB o MongoDB).
2. Añada una **Conexión de datos** al formulario y elija esa conexión en su propiedad **Conexión**.

3. Añada una **Tabla de datos**, elija la conexión en **Fuente de datos** y escriba la **Consulta** y las **Columnas** (`campo|título|ancho`). En MongoDB, la consulta es el mismo JSON del módulo Bases de datos (`find`, `aggregate`, `count` o `distinct`).
4. Para editar la fila elegida, enlace un **Campo de texto** con **Tabla enlazada** y **Campo enlazado**.

Desde el código, `self.Connection1.query(sql, params)` devuelve filas y `self.Connection1.execute(sql, params)` modifica datos (siempre con parámetros `?`, nunca uniendo texto). `self.Treeview1_refresh()` vuelve a cargar la tabla.

Con MongoDB, `query` y `execute` reciben el JSON en texto o como diccionario (`insert`, `update` y `delete` con `execute`), y cada valor `"?"` del JSON toma el siguiente parámetro. Necesita pymongo: si falta, el estudio ofrece **Instalar**.

> ⓘ Las contraseñas no se escriben en el archivo generado: llegan al proceso en variables de entorno `DMSTUDIO_DB_…`. Un archivo exportado las lee del entorno.

Vista HTML: webs de verdad

La **Vista HTML** muestra HTML escrito por su código. Con **Página completa (web)** activada, la página entera es esa vista: sin el marco del formulario, con su propio CSS, como una web normal. Así están hechos los ejemplos **Blog web** y **Tienda web**.

- `self.Site.set_html(html, top=True)` cambia el contenido (`top` vuelve al principio de la página).
- Un enlace o botón con `data-action="post" data-arg="7"` llama al evento **Al pulsar un enlace o enviar un formulario** con un `HtmlAction`: `action.name` es `post` y `action.arg` es `7`.
- Un `<form data-action="buscar">` envía sus campos: se leen con `action.get("q")` o en `action.form`.
- En el escritorio, la vista muestra el HTML sencillo (títulos, párrafos, listas, enlaces y formularios) en un texto; los estilos CSS solo se aplican en la web.

```
import html

class Form1(FormDesign):
    def form_create(self):
        self.Site.set_html('<h1>Hola</h1><a href="#" data-
action="saludar">Saludar</a>')

    def Site_action(self, action):
        if action.name == "saludar":
            nombre = html.escape(action.get("nombre", "mundo"))
            self.Site.set_html("<h1>Hola, %s</h1>" % nombre, top=True)
```

> ⚠ La Vista HTML pinta el HTML tal cual: escape siempre con `html.escape` lo que escriba el visitante. La página solo ejecuta los scripts del propio motor, así que un `<script>` dentro del HTML no se ejecuta.

El **Blog web** y la **Tienda web** traen cuentas de usuario: **Crear cuenta**, **Entrar** y **Salir**. Las contraseñas se guardan con PBKDF2 y sal, nunca en claro, y tras cinco intentos fallidos hay que esperar un minuto. La primera cuenta que se registra es la de administración y ve el **Panel**: en el blog, las entradas, los comentarios y los papeles de los usuarios; en la tienda, las ventas por categoría, los pedidos y el stock. En la tienda, cada cliente ve sus pedidos en **Mi cuenta**.

El proyecto, su icono y el código

El árbol empieza por el nodo **Proyecto**; debajo, la ventana y sus componentes. Al elegirlo, el inspector muestra el nombre, la descripción, el destino, la carpeta (**Ir a la carpeta**) y el **icono de la aplicación**.

- **Icono automático**: todo proyecto con ventana lleva uno sin hacer nada. El estudio lo elige entre sus 500 iconos por las palabras del nombre y de la descripción («Calculadora», «Tienda online», «Obras BIM»…), con un color fijo para ese nombre, y lo guarda en `images/app_icon.png`. Si cambia el nombre, el icono lo sigue.
- **Galería…** muestra los 500 iconos con un buscador (en español o en inglés) y 12 colores. El que elija se queda aunque cambie el nombre; **Automático** vuelve al que pone el estudio.
- **Archivo…** abre el diálogo de archivos: el PNG se copia a `images/` del proyecto.
- **Generar con IA** dibuja el icono a partir de una descripción (con la IA de Ajustes) y lo guarda como PNG de 512 × 512. Antes de usarlo se ve y se puede pedir otra versión.
- Cada destino lo usa: el icono de la ventana (tkinter, Electron, C++), el de la extensión de Chrome o el de la web de Next.js.
- **Subproyectos**: un proyecto puede tener partes con su propio destino (por ejemplo, «Odoo» con «bd» en Python y «front» en Next.js). Se crean con **Añadir subproyecto…**, cuelgan del nodo Proyecto y un

clic los abre en su pestaña; cada uno es una carpeta dentro de la del proyecto.

- En el código, **Ctrl+Espacio** muestra lo que se puede escribir: tras `self.`, `this.` o `Label1->`, los componentes y lo de cada uno; también aparece solo al escribir. **⌘F / Ctrl+F** busca y reemplaza.
- **Ayuda**: clic derecho sobre una palabra del código → **Ayuda** (o **F1**) abre a la derecha un panel con qué es, cómo se escribe y un ejemplo, en Python, JavaScript y C++. Se puede buscar cualquier comando, cambiar de lenguaje, copiar el ejemplo o **Insertar** en la línea del cursor.

Capítulo 4

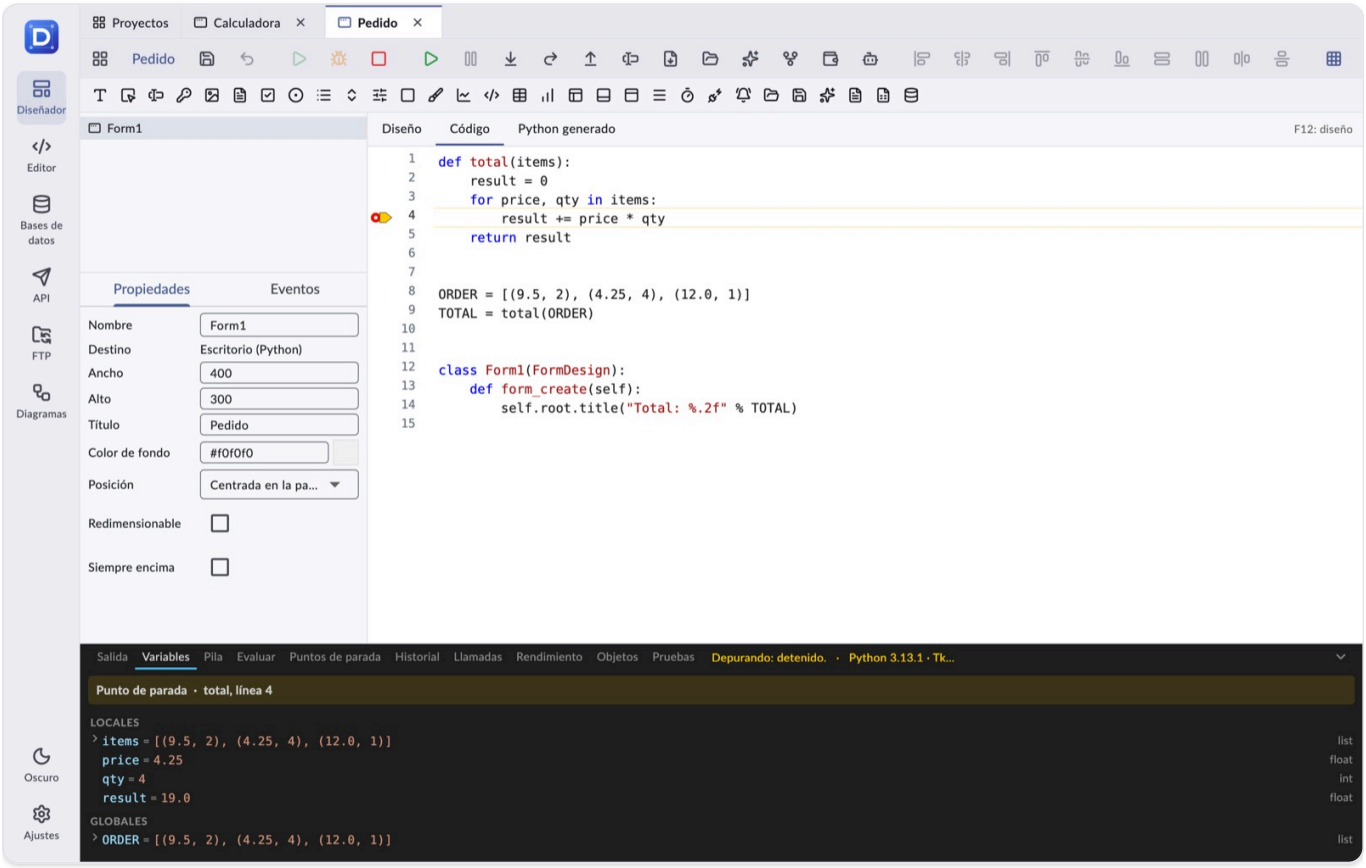
Depurar

Puntos de parada, paso a paso, variables, evaluar, excepciones, historial, rendimiento y pruebas.

Puntos de parada y depurar

El depurador ejecuta su proyecto (de escritorio o web) y lo detiene en las líneas que usted elija de **su código**: el código que genera el diseño no se depura. No hay que instalar nada: usa la biblioteca estándar de Python.

1. En la pestaña **Código**, haga clic a la izquierda de un número de línea para poner un **punto de parada** (un punto rojo); otro clic lo quita. **F5** lo pone en la línea del cursor.
2. Con **clic derecho** sobre el punto se le pone una **condición** (por ejemplo `qty == 4`): solo se detiene cuando es verdadera. Se ve con un punto blanco dentro.
3. Pulse **Depurar** (el bicho naranja, **Mayús+F9**). El programa arranca y, al llegar a un punto, se detiene: la flecha amarilla marca la línea y el panel inferior abre **Variables**.



Detenido en la línea 4 cuando qty vale 4: las variables locales y globales, con las listas y los objetos desplegables.

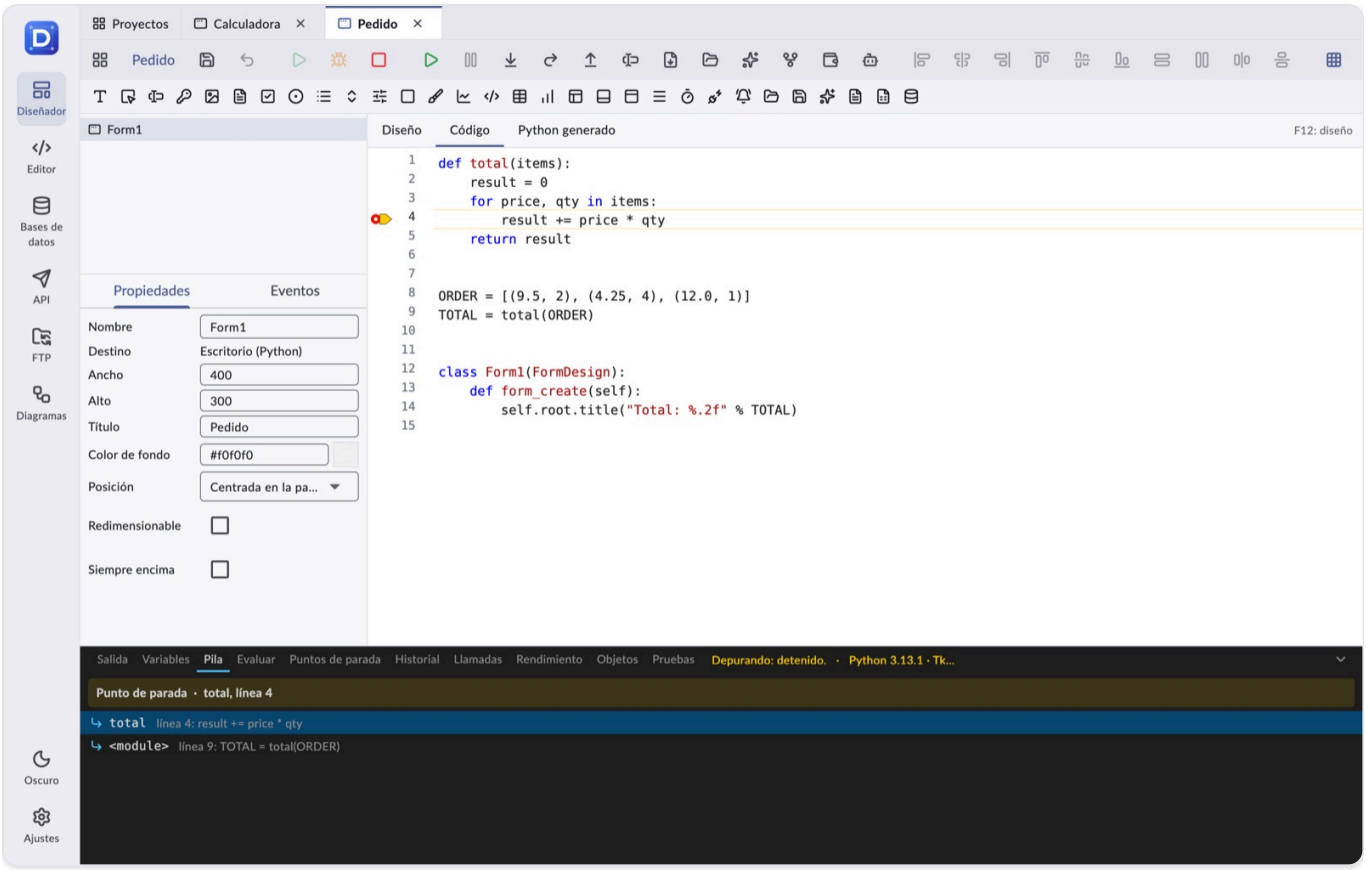
Paso a paso

Botón	Atajo	Qué hace
Continuar	F9	Sigue hasta el siguiente punto de parada
Entrar	F7	Ejecuta la línea y entra en las funciones de su código que llame
Pasar por encima	F8	Ejecuta la línea entera, sin entrar en las funciones
Salir de la función	Mayús+F8	Sigue hasta volver a quien llamó a esta función
Ejecutar hasta el cursor	F4	Sigue hasta la línea donde está el cursor
Pausar		Detiene el programa en la próxima línea de su código que se ejecute
Detener	Ctrl+F2	Termina el programa

Mientras está detenido, la ventana (o la página web) no responde: es normal. Al continuar sigue donde estaba.

Variables, pila y evaluar

- **Variables**: las locales y las globales del marco elegido. Las listas, los diccionarios y los objetos (como `self`) se despliegan para ver su estado.
- **Pila**: la cadena de llamadas hasta la línea actual. Un clic en un marco lo elige: cambian las variables y el editor salta a su línea. Si se detuvo por un error, debajo está la traza completa.
- **Evaluar**: escriba una expresión de Python (`self.total`, `len(items)`) y pulse Intro. También admite asignaciones (`n = 5`) para cambiar un valor antes de seguir.
- **Puntos de parada**: todos los puntos con su línea y su condición; desde aquí se editan, se quitan, y se elige cuándo detenerse en las **excepciones**: nunca, en las no controladas (por defecto: un error en un evento detiene en su línea) o en todas, también las que su código controla con `try`.



La pila: total, llamada desde la línea 9 del módulo.

Historial, llamadas, rendimiento y objetos

- **Historial**: las últimas líneas de su código que se ejecutaron, con la hora y la función. Un clic lleva a la línea.
- **Llamadas**: cada llamada a una función de su código y su vuelta, sangradas por profundidad y con lo que tardó: el flujo del programa.
- **Rendimiento**: por cada función, cuántas veces se llamó y su tiempo total, medio y máximo (sin contar las pausas del depurador), y las líneas que más se ejecutan. Se actualiza solo; **Reiniciar** pone los contadores a cero.
- **Objetos**: cuántos objetos vivos hay de cada clase de su programa (y componentes de tkinter): sirve para ver si algo se crea de más.

Pruebas

La pestaña **Pruebas** ejecuta las pruebas de `unittest` de la carpeta `tests` del proyecto (archivos `test_*.py`). **Crear una prueba de ejemplo** añade `tests/test_form1.py`: las pruebas importan su programa con `from dms_app import Form1` (en una web, `from dms_app import app` y su cliente de pruebas).

```
import unittest

from dms_app import Form1


class TestForm1(unittest.TestCase):
    def setUp(self):
        self.form = Form1()
        self.form.root.withdraw()

    def tearDown(self):
        self.form.root.destroy()

    def test_title(self):
        self.assertTrue(self.form.root.title())
```

Ejecutar pruebas muestra cada prueba en verde, en rojo (con su error al pulsarla) u omitida. **Depurar pruebas** las ejecuta con el depurador: se detienen en los puntos de parada de su código.

Capítulo 5

Electron + React

Aplicaciones de escritorio con Electron y React: el mismo diseñador, con el código en JavaScript.

Crear un proyecto de Electron

Al crear la aplicación, elija **Electron + React**. El Diseñador, el inspector y los eventos son los mismos; cambia el lenguaje del código: **JavaScript**, en el archivo `Form1.js`. La paleta solo muestra los componentes que ya tiene Electron.

> Necesita **Node.js** (nodejs.org). La primera vez que ejecute un proyecto de Electron, el estudio instala Electron y React (unos 100 MB) en su carpeta de datos; después, todos los proyectos los comparten y arrancan al momento.

El código: componentes con propiedades

Cada evento del diseño es un método de la clase `Form1`, que hereda de `FormDesign`. Cada componente es `this.Nombre`, con propiedades sencillas: al cambiarlas, la ventana se vuelve a pintar (con React).

```
class Form1 extends FormDesign {
  form_create() {
    this.List1.items = ["Madrid", "Lima", "Bogotá"];
  }

  Button1_click() {
    this.Label1.text = "Hola, " + this.Entry1.text;
    this.Status.set("Saludado");
  }

  List1_select(event) {
    this.title = "Elegida: " + this.List1.selected;
  }
}
```

Componente	Propiedades y métodos
Todos	`text`, `enabled`, `visible`, `bg`, `fg`, `x`, `y`, `width`, `height`, `focus()`, `configure({...})`

Componente	Propiedades y métodos
Entrada, Texto, Contraseña	`text` (también `value`), `readonly`
Casilla, Opción	`checked`
Lista, Combo	`items`, `index`, `selected`, `selection` (varias), `add(texto)`, `clear()`, `remove(i)`
Tabla de datos	`columns`, `rows` (listas u objetos), `addRow(fila)`, `index`, `selectedRow`, `clear()`
Escala, Barra de progreso	`value`, `maximum`; la barra indeterminada, `start()` y `stop()`
Control de páginas	`page` (la página activa)
Vista HTML	`html`, `setHtml(html, arriba)`; los enlaces y formularios con `data-action` llaman a su evento
Barra de estado	`text`, `icon`, `set(texto, icono)`
Temporizador	`interval`, `start()`, `stop()`
Diálogos Abrir y Guardar	`await execute()` (la ruta o null), `readText(ruta)`, `writeText(ruta, texto)`
La ventana (`this`)	`title`, `message(texto)`, `ask(pregunta)`, `notify(texto)`, `after(ms, función)`, `close()`

El menú es el **menú nativo** del sistema: su evento recibe la opción elegida, como `"Archivo/Abrir…"`. En el código también funciona `require` de Node (`fs`, `path`…).

Ejecutar, depurar y exportar

- **Ejecutar (F9)** genera la aplicación en `build/electron` y la abre en su propia ventana. Lo que escribe `console.log` y los errores (con su línea de `Form1.js`) salen en la pestaña **Salida**.
- **Depurar (Mayús+F9)** la abre con las **herramientas de desarrollo** de Chromium: sus puntos de parada se convierten en `debugger;` en la misma línea, y allí tiene las variables, la pila, la consola, el paso a paso y el rendimiento.
- **Exportar la aplicación de Electron** crea una carpeta con su `package.json`: se abre en cualquier equipo con `npm install` y `npm start`.

Aún no están en Electron: el gráfico, el lienzo, la cinta de opciones, la integración API, el asistente IA, los informes, el icono de bandeja y las conexiones de datos. Tampoco «Pedir a la IA» ni la pestaña Pruebas. Llegarán en próximas versiones.

Capítulo 6

Extensiones de Chrome

La ventana emergente de una extensión, diseñada igual, con el código en JavaScript.

Crear una extensión

Al crear la aplicación, elija **Extensión de Chrome**. El diseño es la **ventana emergente** que se abre al pulsar el icono de la extensión (800 × 600 como mucho) y el código es el mismo JavaScript de Electron, con los mismos componentes (todos menos el menú).

El estudio genera una extensión **Manifest V3** completa en `build/extension`: `manifest.json`, `popup.html` y React empaquetado dentro (una extensión no puede cargar código de fuera). Necesita **Node.js** la primera vez, como Electron.

Lo propio de una extensión

Desde el código	Qué hace
`await this.storage.get(clave, siNoHay)`	Lee un dato guardado (`chrome.storage`)
`await this.storage.set(clave, valor)`	Guarda un dato (texto, número, lista u objeto)
`await this.currentTab()`	La pestaña activa: `{ id, url, title }`
`this.openTab(url)`	Abre una dirección en una pestaña nueva
`await this.runInPage(función, ...args)`	Ejecuta la función dentro de la página abierta y devuelve lo que devuelva
`this.badge(texto, color)`	Pone texto en la insignia del icono

```
class Form1 extends FormDesign {
  async Count_click() {
    const text = await this.runInPage(() => document.body.innerText);
    const words = text.split(/\s+/).filter(Boolean).length;
    this.Words.text = words + " palabras";
    this.badge(String(words));
  }
}
```

> ⓘ La función de `runInPage` corre en la página, no en la extensión: solo ve lo que hay en la página y lo que reciba en `args`. Las páginas de Chrome (`chrome://…`) no se pueden leer.

Probar, cargar en Chrome y publicar

- **Ejecutar (F9)** abre una **vista previa** de la ventana emergente: los datos se guardan en la vista previa y la pestaña activa es una de ejemplo. **Depurar** la abre con las herramientas de desarrollo.
- **Cargar en Chrome** prepara la extensión, copia su carpeta y abre `chrome://extensions`: active el **Modo de desarrollador**, pulse **Cargar extensión sin empaquetar** y elija esa carpeta. Tras cada cambio, guarde y pulse ↻ en la tarjeta de la extensión.
- **Exportar la extensión** crea su carpeta y un `.zip`, que es lo que se sube a la Chrome Web Store.

Capítulo 7

Web con Next.js

Páginas web de Next.js: el diseño es la página y el código, JavaScript con React.

Una web con pantallas (como Flutter)

Al crear el proyecto, **Web (Next.js)** usa el mismo editor de pantallas que Flutter: pantallas con su árbol de widgets, propiedades, menú y las 20 plantillas. Cada pantalla es una página de la web (la primera es `/`; las demás, `/su-nombre`), hecha en React con HTML y CSS adaptables: el contenido se centra en escritorio y el menú pasa de barra lateral a barra de abajo en el móvil. La lógica va en `lib/app.js`: una clase `App` que llama a `this.notify()` al cambiar.

```
export class App extends DmsApp {
  tareas = [];
  nueva = "";

  agregar() {
    this.tareas.unshift({ titulo: this.nueva, hecha: false });
    this.nueva = "";
    this.notify();
  }
}
```

- Las propiedades se escriben igual que en Flutter: `${…}` en los textos, `=expresión` (de JavaScript) y las acciones `go:Pantalla(item)`, `home:Pantalla`, `back` o código. `.isEmpty` y `.isNotEmpty` también valen.
- En el editor, la vista previa se ve en el navegador de escritorio o en el móvil (el botón de la paleta).
- **Ejecutar (F9)** arranca `next dev` y abre el navegador; al guardar, la página se recarga.
- Ejemplo: **Tienda web**, con login, catálogo, ficha, carrito y pago.
- Los proyectos de antes siguen funcionando como **Web (Next.js, formulario)**.

Una web con servidor

En **Web (Next.js)**, el código puede tener una parte que se ejecuta en el servidor: todo lo que va después de una línea `// @server` es `lib/server.js`. Allí van las claves, las llamadas a APIs de otros sitios y lo que no debe llegar al navegador. Cada función que exporta se llama desde la app con `await server.nombre({ … })` (el estudio genera la ruta `app/api/[fn]/route.js`) y devuelve lo que ella devuelva; si devuelve un `Response`, sale tal cual (para descargas o datos que no son JSON).

```
import { DmsApp, server } from "./dms";

export class App extends DmsApp {
  precio = null;

  async start() {
    this.precio = await server.precio({ moneda: "EUR" });
    this.notify();
  }
}

// @server: lo que sigue se ejecuta en el servidor.
export async function precio({ moneda }) {
  const r = await fetch("https://api.ejemplo.com/precio?m=" + moneda, {
    headers: { authorization: "Bearer " + process.env.API_KEY },
  });
  return r.json();
}
```

- La parte del servidor es otro módulo: no ve las variables de la app (y la app no ve las suyas). Lo que necesiten las dos se pasa en los argumentos o en lo que devuelve.
- Las claves van en `.env.local` (en `build/nextjs` al ejecutar desde el estudio) o en las variables de entorno de Vercel, y se leen con `process.env`.
- `serverUrl("nombre", { … })` da la dirección de una función, para un `fetch` propio, un enlace o una descarga.
- Todo lo que exporta `lib/server.js` se puede llamar desde fuera: compruebe lo que llega en los argumentos.
- Ejemplos: **El mundo en directo (Next.js)**, que junta en el servidor los aviones, los barcos y los trenes de cuatro APIs públicas y los dibuja en un Lienzo, y **Velocidad de internet (Next.js)**, cuyo servidor devuelve un `Response` con bytes al azar para medir la bajada y cuenta los que recibe para la subida, y **Buscador de vuelos (Next.js)**, que guarda el token de Travelpayouts en el servidor (`TRAVELPAYOUTS_TOKEN`).

Una web de React (Vite)

- **Web (React)** usa el mismo editor de pantallas y la misma lógica (`src/lib/app.js`), pero genera una web de una sola página con React y Vite: las rutas van tras `#` (`#/catalogo`) y `npm run build` deja en `dist/` archivos estáticos para cualquier hosting, sin servidor.
- **Ejecutar (F9)** arranca Vite y abre el navegador; al guardar, la página se recarga. Vite se instala solo la primera vez (unos 40 MB, compartidos).
- Ejemplo: **Tienda web (React)**, la misma tienda que en Next.js y React Native.

Una web en HTML

- **Web (HTML)** usa el mismo editor de pantallas y la misma lógica (`app.js`), pero genera HTML, CSS y JavaScript sin nada más: no se compila, no necesita Node.js ni dependencias. `index.html` carga `dms.js` (un motor pequeño que dibuja la pantalla de la ruta tras `#`), `app.js` y `screens.js`.
- La clase `App` es la misma que en Next.js y React; las líneas `import` y `export` se ignoran, así que la lógica pasa de un destino a otro sin cambios.
- **Ejecutar (F9)** sirve la carpeta con el servidor propio del estudio y abre el navegador; al guardar, la página se recarga.
- **Exportar** deja la carpeta lista: abra `index.html` o súbala a cualquier hosting.
- Ejemplo: **Tienda web (HTML)**, la misma tienda que en Next.js y React.
- **Lienzo**: un componente del editor de pantallas para dibujar con código. **Dibujar** llama a un método de `App` con `ctx` (el contexto 2D del navegador; en Flutter, el `Canvas`), `w` y `h` cada vez que la app avisa con `notify()`; **Al tocar** recibe `x` e `y`. Se ajusta al ancho y al alto de la pantalla con su **Proporción**. Funciona en HTML, React, Vue, Next.js y Flutter; en React Native, de momento, enseña un aviso.
- Ejemplo: **Dominó doble 9 (web)**, una pantalla con un Lienzo: toda la partida va en `app.js` y se juega en el navegador.

Una web de Vue (Vite)

- **Web (Vue)** usa el mismo editor de pantallas y la misma lógica (`src/lib/app.js`), pero cada pantalla es un componente de Vue (`src/screens/Pantalla.vue`) con su plantilla: `v-for` para las listas, `v-if` para lo que se muestra o no y `@click` para las acciones.
- `this.notify()` sube una versión que leen las pantallas, y Vue las vuelve a dibujar; la clase `App` es la misma que en Next.js, React y HTML.
- Las rutas van tras `#` y `npm run build` deja en `dist/` archivos estáticos para cualquier hosting.
- **Ejecutar (F9)** arranca Vite y abre el navegador; al guardar, la página se recarga. Vite y Vue se instalan solos la primera vez (compartidos con React).
- Ejemplo: **Tienda web (Vue)**, la misma tienda que en las otras webs.

Una web con servidor en Go

- **Web (Go)** usa el mismo editor de pantallas y la misma lógica (`app.js`), servidas por un servidor de Go (`main.go`, solo la biblioteca estándar) que da una **API REST** con los datos guardados en archivos JSON (`data/<colección>.json`).
- Desde la lógica: `await api.list("pedidos")`, `api.list("pedidos", { cliente: "Ana" })` (filtro), `api.get(col, id)`, `api.add(col, registro)` (devuelve el registro con `id` y `created_at`), `api.update(col, id, cambios)` y `api.remove(col, id)`.

- **Ejecutar (F9)** compila el servidor y abre el navegador; al guardar, la página se recarga. Los datos van en la carpeta `data` del proyecto y siguen ahí al reiniciar.
- **Exportar** deja el proyecto de Go: `go run .` o `go build` (un solo ejecutable con la web dentro). Necesita Go (go.dev); en macOS, con Go anterior a 1.22, el estudio enlaza con el enlazador del sistema.
- Ejemplo: **Tienda web (Go)**: los pedidos se guardan en el servidor y la numeración sigue al reiniciarlo.

Crear una página de Next.js

Al crear la aplicación, elija **Web (Next.js)**. El diseño es la página y el código es el mismo JavaScript de Electron (corre en el navegador), con los mismos componentes (todos menos el menú). El estudio genera un proyecto de Next.js (app router) en `build/nextjs`.

- **Ejecutar (F9)** arranca el servidor de desarrollo de Next.js y abre el navegador cuando está listo. Next.js, React y Electron se instalan una sola vez y los comparten todos los proyectos.
- `this.storage` guarda datos en el navegador; `this.currentTab()` devuelve la propia página y `this.runInPage(función)` la ejecuta en ella.
- **Depurar** convierte sus puntos de parada en `debugger;`: abra las herramientas del navegador (F12) y se detendrá en ellos.
- **Exportar el proyecto de Next.js** crea una carpeta lista para `npm install` y `npm run dev`, o para publicarla en Vercel.

Capítulo 8

C++ con FLTK

Aplicaciones de escritorio nativas en C++: el mismo diseñador, compiladas con FLTK.

Crear una aplicación de C++

Al crear la aplicación, elija **C++ (FLTK)**. El código va en `Form1.cpp`: la clase `Form1` hereda de `FormDesign` y cada evento es un método con `override`. Cada componente es un puntero a una clase de FLTK con métodos sencillos (`text`, `checked`, `items`…), sin perder la API de FLTK.

```
class Form1 : public FormDesign {
public:
  void Button1_click() override {
    Label1->text("Hola, " + Entry1->text());
    Status->text("Saludado");
  }

  void Scale1_change(double value) override {
    Bar1->value(value);
  }
};
```

Componente	Métodos
Etiqueta, Botón, Barra de estado	`text()`, `text(s)`, `enabled(bool)`, `visible(bool)`, `bg(color)`, `fg(color)`
Entrada, Contraseña, Texto	`text()`, `text(s)`
Casilla, Opción	`checked()`, `checked(bool)`
Lista, Combo	`items({...})`, `add(s)`, `index()`, `index(i)`, `selected()`
Escala, Barra de progreso	`value()`, `value(v)`
Control de páginas	`page()`, `page(i)`
Temporizador	`Timer1.start()`, `Timer1.stop()`
Diálogos Abrir y Guardar	`OpenFile.execute()` (la ruta, o "" si se cancela)

Componente	Métodos
La ventana	`title(s)`, `message(texto)`, `ask(pregunta)`, `close()`; y `str(número)`, `readText(ruta)`, `writeText(ruta, texto)`

Compilar, ejecutar y exportar

- **Ejecutar (F9)** compila con el compilador de C++ del equipo y FLTK, y abre la aplicación. Los errores salen en la pestaña Salida con su línea de `Form1.cpp`.
- Si falta FLTK o el compilador, el estudio lo dice y ofrece **Instalar** (en macOS, con Homebrew: `brew install fltk`).
- **Depurar** compila con información de depuración (`-g`) para abrir `build/cpp/app` con lldb o gdb.
- **Exportar el proyecto de C++** crea una carpeta con las fuentes y un `Makefile`: `make run`.

Capítulo 9

Pascal con Lazarus

Aplicaciones de escritorio nativas en Object Pascal, con los componentes de la LCL.

Crear una aplicación de Pascal

Al crear la aplicación, elija **Pascal (Lazarus)**. El código va en la unidad `Form1.pas`: la clase `TForm1` hereda de `TFormDesign` y cada evento es un método que se declara en la clase con `override` y se escribe en `implementation`. El estudio añade las dos partes al crear un evento. Los componentes son los de la LCL de siempre.

```
type
  TForm1 = class(TFormDesign)
  public
    procedure Button1_click; override;
    procedure Scale1_change(Value: Double); override;
  end;

implementation

procedure TForm1.Button1_click;
begin
  Label1.Caption := 'Hola, ' + Entry1.Text;
  Status.SimpleText := 'Saludado';
end;

procedure TForm1.Scale1_change(Value: Double);
begin
  Bar1.Position := Round(Value);
end;
```

Componente	Clase de la LCL y lo más usado
Etiqueta	`TLabel`: `Caption`, `Font`, `Color`
Botón	`TButton` (o `TDmsButton` si tiene color o es alto): `Caption`, `Enabled`
Entrada, Contraseña	`TEdit`: `Text`, `Clear`, `SetFocus`
Texto	`TMemo`: `Text`, `Lines.Add`, `Lines.LoadFromFile`
Casilla, Opción	`TCheckBox`, `TRadioButton`: `Checked`

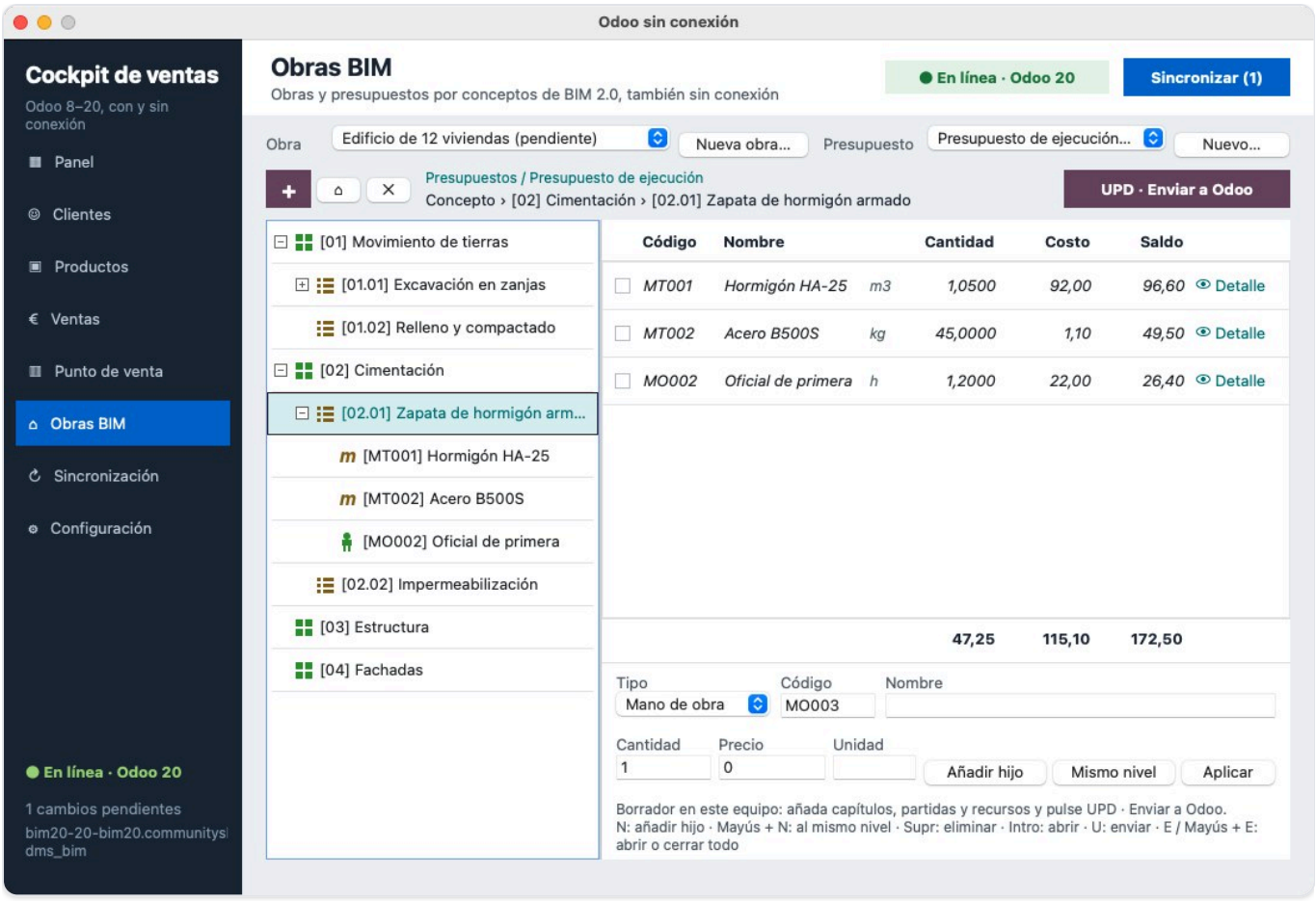
Componente	Clase de la LCL y lo más usado
Lista, Combo	`TListBox`, `TComboBox`: `Items`, `ItemIndex`
Escala, Barra de progreso	`TTrackBar`, `TProgressBar`: `Position`, `Max`
Marco, Control de páginas	`TPanel`, `TPageControl` (cada página, `Pages1_page0`…)
Barra de estado	`TStatusBar`: `SimpleText`
Menú, Temporizador	`TMainMenu`, `TTimer`: `Enabled`, `Interval`
Diálogos Abrir y Guardar	`TOpenDialog`, `TSaveDialog`: `Execute`, `FileName`
La ventana y el estudio	`Caption`, `Close`, `ShowMessage(texto)`, `Ask(pregunta)`, `NumToStr(x)`, `ReadText`, `WriteText`, `LastKey` (en el evento de teclado)

Compilar, ejecutar y exportar

- **Ejecutar (F9)** compila con `lazbuild` y abre la aplicación. Los errores salen en la pestaña Salida con su línea de `Form1.pas`, como `Form1.pas(22,3) Error: Identifier not found`.
- Si falta Lazarus, el estudio lo dice y ofrece **Instalar** (en macOS, con Homebrew: `brew install fpc` y `brew install --cask lazarus`). En un Mac con Apple Silicon se compila para arm64.
- Lo que escribe `WriteLn` sale en la pestaña Salida. Para depurar paso a paso, abra `build/pascal/app.lpi` en el IDE de Lazarus.
- **Exportar el proyecto de Lazarus** crea una carpeta con `app.lpi` y las unidades: se abre en Lazarus o se compila con `lazbuild app.lpi`.

Ejemplo: Odoo sin conexión y obras BIM

El ejemplo **Odoo sin conexión (Pascal)** es una aplicación nativa que trabaja con Odoo 8 a 20 por JSON-RPC, con y sin conexión: panel, clientes y proveedores, productos (que se crean también sin conexión), ventas y compras con sus pasos (**Confirmar**, **Facturar** y **Registrar pago**, que Odoo repite al sincronizar), punto de venta y, si la base de datos de Odoo tiene **BIM 2.0** (el módulo `base_bim_2`), la página **Obras BIM** para hacer obras y presupuestos por conceptos. Se descarga ya compilado en la página **Demos**.



Obras BIM, como el árbol de conceptos de BIM 2.0: a la izquierda el árbol; a la derecha, lo que hay dentro de la partida elegida.

1. En **Configuración**, escriba la dirección de Odoo, la base de datos, el usuario y la contraseña y pulse **Conectar y descargar**. Si Odoo tiene BIM 2.0, aparece **Obras BIM** en el menú, detrás de **Punto de venta**.
2. Elija una obra de Odoo o cree una con **Nueva obra…** (nombre y, si quiere, cliente). Luego cree su presupuesto con **Nuevo…**.
3. En el árbol, sin elegir nada (o con ⌂), añada los capítulos; elija un capítulo para añadir sus partidas y una partida para añadir sus recursos: mano de obra, equipo, material, subcontrata o función. El editor de debajo propone el tipo y el código (01, 01.01, MO001…); escriba el nombre, la cantidad, el precio y la unidad y pulse **Añadir hijo**, **Mismo nivel** o **+**.
4. La tabla muestra lo que hay dentro de lo elegido con su **Cantidad**, **Costo** y **Saldo**, y los totales. Elija una fila para cambiarla con **Aplicar**; **Detalle**, el doble clic o Intro entran en ella.
5. Pulse **UPD · Enviar a Odoo**: con conexión se sube al momento; sin ella, espera en **Sincronización** y se envía al volver la red. Odoo recalcula los importes (UPD) y el presupuesto queda con su código PSU….

Tecla	En el árbol o en la tabla
N	Añadir un hijo de lo elegido
Mayús + N	Añadir al mismo nivel

Tecla	En el árbol o en la tabla
Supr	Eliminar (con lo que lleva dentro)
Intro	Abrir (entrar en el concepto)
U	Enviar el presupuesto a Odoo (UPD)
E / Mayús + E	Abrir o cerrar todo el árbol

Los importes se calculan como en BIM: un recurso, cantidad × precio; una partida, su cantidad × la suma de sus recursos (o su precio, si no tiene); un capítulo, la suma de lo que lleva dentro. Los capítulos van en el primer nivel, las partidas dentro de un capítulo y los recursos dentro de una partida.

Sin duplicados: la obra lleva su referencia DMS-… en las observaciones, el presupuesto en su origen y cada concepto en su «id_import». Antes de crear algo se busca en Odoo, así que reenviar no duplica nada y un envío cortado sigue donde se quedó. Los presupuestos que ya están en Odoo se ven en el mismo árbol (se cambian en Odoo). Probado con BIM 2.0 en Odoo 18 y 20.

Capítulo 10

Apps móviles con Flutter

Pantallas al estilo de FlutterFlow, lógica en Dart y un simulador de teléfono propio.

Crear una app de Flutter

Al crear el proyecto, elija **Flutter (móvil)**. La pestaña **Diseño** es un editor al estilo de FlutterFlow: a la izquierda, las **pantallas** y el **árbol de widgets** de la elegida; en el centro, la **paleta** y el **teléfono** con la vista previa; a la derecha, las **propiedades** del widget elegido (o de la pantalla y de la app).

- Un widget de la paleta se añade dentro del elegido si es una columna, una fila o una lista; si no, detrás de él. Se elige tocándolo en el teléfono o en el árbol; **Subir**, **Bajar**, **Duplicar** y **Eliminar** están sobre el árbol.
- Widgets: columna, fila, lista, tarjeta, contenedor, margen, centrado, expandido, texto, botón, botón de icono, campo de texto, fila de lista, casilla, interruptor, etiqueta, icono, imagen, progreso, separador y espacio.
- **+** en Pantallas abre la galería: una pantalla **en blanco** o una de las **20 plantillas** más usadas (inicio de sesión, registro, recuperar contraseña, bienvenida, inicio, lista, detalle, perfil, editar perfil, ajustes, notificaciones, mensajes, conversación, búsqueda, carrito, pago, pedido confirmado, panel, contacto y ayuda), con su vista previa. Sus botones escriben en la Salida (`debugPrint`) hasta que se enlazan con App.
- Cada pantalla tiene nombre, título, barra superior, un botón flotante y, si se abre con algo (por ejemplo, un envío), el tipo que **recibe**.

La lógica y los enlaces

La pestaña **Código** es `lib/app.dart`: una clase `App` con los datos y los métodos de la app. Las pantallas la usan como `app` y se vuelven a dibujar cuando llama a `notifyListeners()`. **Dart generado** muestra `lib/main.dart`, que el estudio escribe a partir de las pantallas.

```
class App extends ChangeNotifier {
  final tareas = <Tarea>[];
  String nueva = '';

  void agregar() {
    tareas.insert(0, Tarea(nueva));
    nueva = '';
    notifyListeners();
  }
}
```

En las propiedades	Ejemplo
Un texto con valores	`${item.titulo}` · `Hola, ${app.nombre}`
Una expresión (con =)	`=item.hecha` · `=app.color(item.estado)`
Una acción	`app.agregar()` · `go:Detalle(item)` · `app.crear(); back`
Una lista enlazada	Elementos: `=app.tareas`; su primer hijo se repite con `item`
Un campo de texto	Guarda en: `app.nueva`
Visible si	`=app.tareas.isNotEmpty`
Una acción con condición	`if (app.entrar()) home:Catalogo`
Botones de la barra	`shopping_cart|go:Carrito|${app.unidades}` (icono, acción e insignia)

Escritorio

- Elija **Flutter (escritorio)** al crear el proyecto: el mismo editor, con la vista previa en una ventana de macOS.
- **Menú** (propiedades de la app): una línea por pantalla, `Pantalla|icono|texto`. En una ventana ancha es una barra lateral (`NavigationRail`); en una estrecha, la barra de abajo. También sirve en las apps móviles.
- **Ejecutar (F9)** abre la app en su ventana nativa (macOS, Windows o Linux) y al guardar se vuelve a cargar.
- Ejemplo: **Inventario**, con panel, productos, ficha con existencias y alta.

FalconDatabase y Firebase

En las propiedades de la app (sin ningún widget elegido), **Base de datos → Conectar con** elige **FalconDatabase** (servidor, referencia del proyecto y clave anon) o **Firebase** (ID del proyecto y clave web). El estudio genera `lib/db.dart` con la misma forma para los dos, y `App` lo usa con `import 'db.dart';`.

```
Future<bool> entrar() async {
  try {
    await db.signIn(email, clave);
    notas = await db.table('notas').list(order: 'id.desc');
    notifyListeners();
    return true;
  } on DbError catch (e) {
    error = e.message;
    notifyListeners();
    return false;
  }
}
```

- `db.signIn(correo, clave)`, `db.signUp(…)`, `db.signOut()` y `db.user`: la sesión.
- `db.table('t').list(where: {…}, order: 'campo.desc', limit: 20)`, `get(id)`, `insert({…})`, `update(id, {…})` y `delete(id)`. En `where` vale un valor o un operador como `gte.10`.
- Las acciones pueden esperar: `if (await app.entrar()) home:Notas`.
- Solo va dentro de la app la clave pública (anon o la clave web); las reglas de acceso las pone la base de datos.
- Ejemplo: **Notas en la nube**, con login y la tabla `notas` de FalconDatabase.

El simulador y exportar

- **Ejecutar (F9)** compila la app con Flutter para la web y la abre en el **simulador del estudio**: un marco de teléfono (iPhone, Pixel, Galaxy o iPad mini) que se puede girar. No hace falta Xcode ni Android.
- Al **guardar**, la app se vuelve a compilar y el simulador se recarga solo.
- Lo que escribe `print` o `debugPrint` sale en la pestaña Salida, y los errores, con su línea de `lib/app.dart`.
- Los paquetes de pub.dev (por ejemplo, `shared_preferences: ^2.3.0`) se añaden en las propiedades de la app.
- **Exportar el proyecto de Flutter** deja `pubspec.yaml`, `lib/main.dart` y `lib/app.dart`: con `flutter create .` y `flutter run` se abre en el teléfono, en Android Studio o en VS Code.
- Ejemplos: **Tienda con login** (login, catálogo con búsqueda y categorías, ficha, carrito con cantidades y pago), **Envíos** (resumen por estado, lista con filtro, detalle con seguimiento y alta) y **Tareas** (añadir,

marcar, filtrar y borrar, guardadas en el teléfono).

Capítulo 11

Apps móviles con React Native

El editor de pantallas con componentes nativos de React Native (Expo) y la lógica en JavaScript.

Crear una app de React Native

Al crear el proyecto, elija **React Native (móvil)**: el mismo editor de pantallas que Flutter (pantallas, árbol de widgets, propiedades, menú y las 20 plantillas). Cada pantalla se genera con los componentes de React Native (`View`, `Text`, `Pressable`, `TextInput`, `ScrollView`, `Switch`…) y la lógica va en `lib/app.js`, la misma clase `App` que en la web de Next.js (con `this.notify()`).

- **Ejecutar (F9)** arranca Expo y abre la app en el simulador de teléfono del estudio; al guardar se recarga. La primera vez instala React Native y Expo (unos 340 MB, compartidos por todos los proyectos).
- En la Salida aparece la dirección `exp://…` para abrir la app en su teléfono con **Expo Go** (en la misma red).
- **Exportar el proyecto de React Native (Expo)** deja `package.json`, `App.js`, `screens/` y `lib/`: `npm install` y `npx expo start`.
- Ejemplo: **Tienda (React Native)**, con login, catálogo, ficha, carrito y pago.

Capítulo 12

Aplicaciones de consola

Programas sin ventana en Python, C, C++ o Pascal, con la entrada y la salida en la pestaña Salida.

Crear un programa de consola

Al crear el proyecto, elija **Consola** y el lenguaje: **Python**, **C**, **C++** o **Pascal**. No hay diseño: el proyecto es un solo archivo (`main.py`, `main.c`, `main.cpp` o `main.pas`) que se edita en la pestaña **Código**, con resaltado, autocompletado y ayuda (F1).

```
name = input("¿Cómo se llama? ")
print(f"Hola, {name}.")
```

- **Ejecutar (F9)** compila si hace falta y arranca el programa. Lo que escribe (`print`, `printf`, `std::cout`, `WriteLn`) sale en la pestaña **Salida**.
- Cuando el programa pide algo (`input`, `scanf`, `std::cin`, `ReadLn`), escriba la respuesta en el campo de abajo de la Salida y pulse **Intro**: aparece detrás de la pregunta, como en un terminal.
- C y C++ se compilan con el compilador del equipo; Pascal, con Free Pascal (`fpc`), sin Lazarus. Los errores salen con su línea, como `main.c:2:3: error: …`.
- **Exportar el programa** guarda el archivo tal cual.

Ejemplos

- **Gestión de animales (consola)**: en Python, con menú para dar de alta y de baja animales y llevar sus vacunas (las próximas dosis y las pendientes de los próximos 30 días). Guarda los datos en `animales.json`, en la carpeta del proyecto.
- **Adivina el número**, en Python, C, C++ y Pascal: el programa piensa un número entre 1 y 100 y usted lo adivina con las pistas «Más alto» y «Más bajo».

Capítulo 13

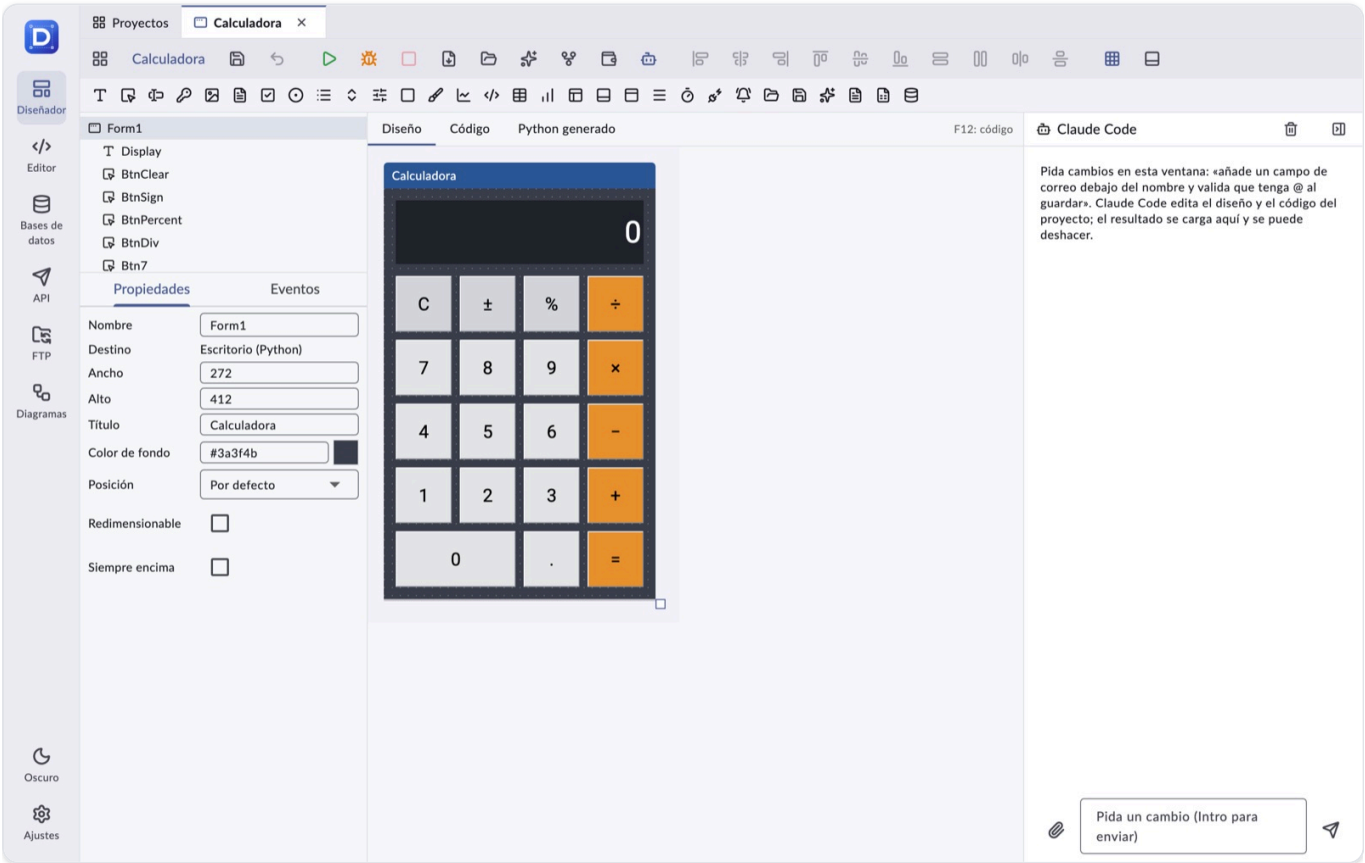
IA en el Diseñador

Chat con Claude Code y «Pedir a la IA».

Chat con Claude Code

El botón **Chat con Claude Code** (el robot de la barra) despliega un panel a la derecha. Describa el cambio y pulse Intro (Mayús+Intro hace un salto de línea).

- Usa la Claude Code instalada en su equipo y su sesión: no necesita clave de API.
- Antes de cada mensaje el estudio guarda el proyecto. Claude Code lee y edita el diseño (`project.dmsproj`) y el código de la ventana, sin terminal.
- Al terminar, el diseño y el código nuevos se cargan en el Diseñador como un solo paso que se puede deshacer.
- La conversación sigue entre mensajes; la papelera empieza una nueva.
- El clip 📎 adjunta imágenes (un boceto, una captura) o archivos de texto: Claude Code los lee sin que entren en el proyecto.



El panel de Claude Code, a la derecha del formulario.

Pedir a la IA

Pedir a la IA usa el proveedor configurado en **Ajustes** (Claude por defecto) y tiene tres modos:

- **Cambiar**: modifica la ventana actual («Añade un campo de teléfono y alinea los botones»).
- **Ventana nueva**: diseña una ventana desde cero; puede **Adjuntar un boceto** (una imagen).
- **Código de** un componente: con un componente seleccionado, escribe el método de su evento principal.

La respuesta llega como **Propuesta de la IA**: se revisa antes de aplicarla y, una vez aplicada, se deshace en un paso.

Presupuesto de IA

El botón de la cartera (**Presupuesto de IA**) muestra lo gastado en Claude en el proyecto: por herramienta (Claude Code, Claude implementa, Pedir a la IA, chat), por modelo y por día, con las últimas llamadas y sus tokens.

- Se puede poner un **Tope (US$)**, **Mensual** o **Total**. Avisa al 80 % y, con **Bloquear al agotarse**, la IA deja de responder en ese proyecto hasta que se suba.
- Los costes usan la tarifa de la API de Anthropic. Con una suscripción de Claude, el de Claude Code es su equivalente en API, no lo que se cobra.

GitHub

- El botón **GitHub** del Diseñador conecta su cuenta con un token (se guarda solo en este equipo).
- **Publicar en GitHub** crea el repositorio (privado o público), hace el primer commit y lo sube.
- Ya publicado: lista los cambios, pide el **Mensaje del commit** y **Confirmar y subir**; también **Traer cambios** y **Abrir en GitHub**.

Capítulo 14

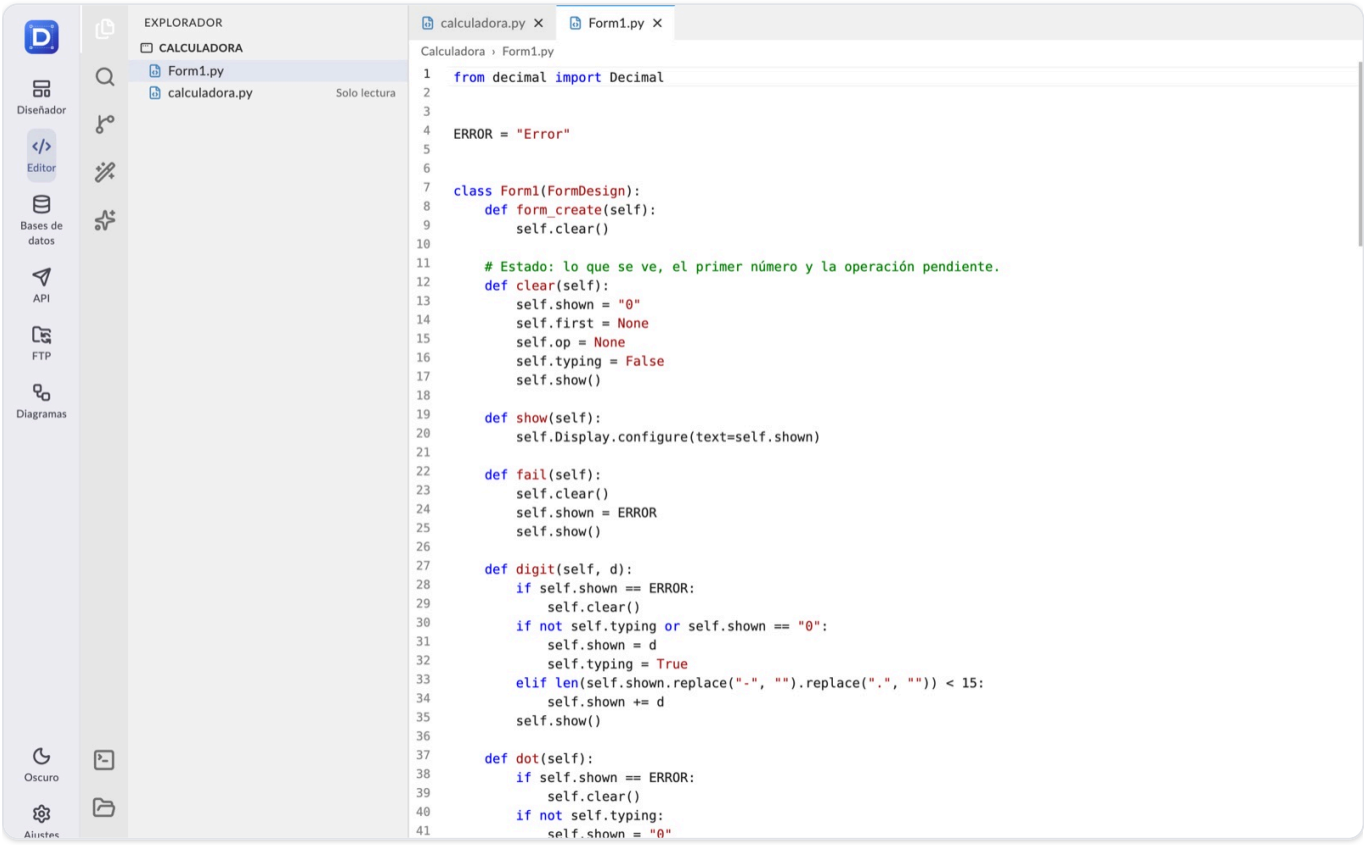
El Editor

Editor de código con explorador, búsqueda, terminal, git e IA.

Abrir una carpeta

Al entrar sin carpeta se ofrecen **Abrir carpeta…**, **Abrir DMarlon Studio** (la carpeta de documentos del estudio), **Clonar repositorio…** y la lista de **Recientes**.

Si hay un proyecto abierto en el Diseñador, el Editor muestra su código y su Python generado como **Proyecto abierto en el Diseñador de interfaz**: lo que escriba aquí es el mismo texto que en la pestaña Código.



El Editor: barra de actividad, explorador y pestañas.

Explorador y pestañas

- La barra de actividad tiene **Explorador**, **Buscar**, **Control de código**, **Claude implementa** y **Chat con la IA**, y abajo la terminal y **Abrir carpeta**.
- El explorador permite **Nuevo archivo**, **Nueva carpeta**, **Renombrar**, **Eliminar**, **Copiar la ruta**, **Abrir en la terminal** y **Contraer todo**.

- Las pestañas tienen resaltado de sintaxis para decenas de lenguajes y números de línea. ⌘S / Ctrl+S guarda; al cerrar algo sin guardar se pregunta.
- Lo que no guarde se conserva al cambiar de módulo.

Buscar y reemplazar

- **Buscar en el archivo** con **Reemplazar** y **Reemplazar todo**.
- El panel **Buscar** recorre todos los archivos de la carpeta, con **Distinguir mayúsculas** y **Expresión regular**; un clic abre el archivo en la línea.

Terminal

⌘J / Ctrl+J muestra u oculta la terminal, que ejecuta órdenes en la carpeta abierta (`git status`, `python3 app.py`, `npm install` …). Las flechas arriba y abajo recorren el historial y **Detener** corta la orden en marcha.

Control de código (git)

- **Cambios**: archivos modificados con su diff; escriba el **Mensaje del commit** y use **Confirmar todo** o **Confirmar y subir**. También **Corregir el último commit** y **Deshacer el último commit**.
- **Subir (push)**, **Traer (pull)** y **Buscar cambios en el remoto (fetch)**, con lo pendiente en cada sentido.
- **Historial** de la rama o de **Todas las ramas**: ver el commit, **Revertir este commit**, **Aplicar en la rama actual (cherry-pick)**, crear una rama o una etiqueta desde él y **Volver la rama a este commit…**.
- **Ramas** locales y remotas: crear, cambiar, **Fusionar**, renombrar y eliminar (también del remoto).
- **Guardar los cambios aparte (stash)** para cambiar de rama y recuperarlos después.
- **Remotos…**, **Inicializar repositorio**, **Clonar repositorio…** y **Publicar en GitHub…** (público o privado) con su **Cuenta de GitHub** (un token con permiso «repo»).

Claude implementa

Claude implementa pide un cambio a Claude a través de la API de Anthropic (Claude Opus 5.5 por defecto). Claude explora la carpeta abierta (lista, lee y busca archivos) y propone cambios en uno o varios archivos.

- Nada se escribe hasta que usted lo aplica: la lista **Cambios propuestos** muestra cada archivo con sus líneas añadidas y quitadas; un clic abre el diff con **Aplicar** y **Descartar**.

- **Aplicar todo** escribe todos los cambios; si un archivo cambió en el disco mientras Claude trabajaba, se avisa en lugar de pisarlo.
- Necesita la clave de Anthropic en **Ajustes → Inteligencia artificial**.
- El clip 📎 adjunta imágenes y archivos de texto o código a la petición; el botón de la cartera abre el **Presupuesto de IA** de la carpeta.

Chat con la IA

El **Chat con la IA** responde sobre el archivo abierto con el proveedor de **Ajustes**. Si propone el archivo completo, **Aplicar al archivo** lo pone en la pestaña (sin guardar) y **Copiar el código** lo copia.

En los tres chats Intro envía, Mayús+Intro salta de línea y el clip 📎 adjunta imágenes (PNG, JPG, GIF o WebP de hasta 5 MB) y archivos de texto (hasta 300 KB).

Capítulo 15

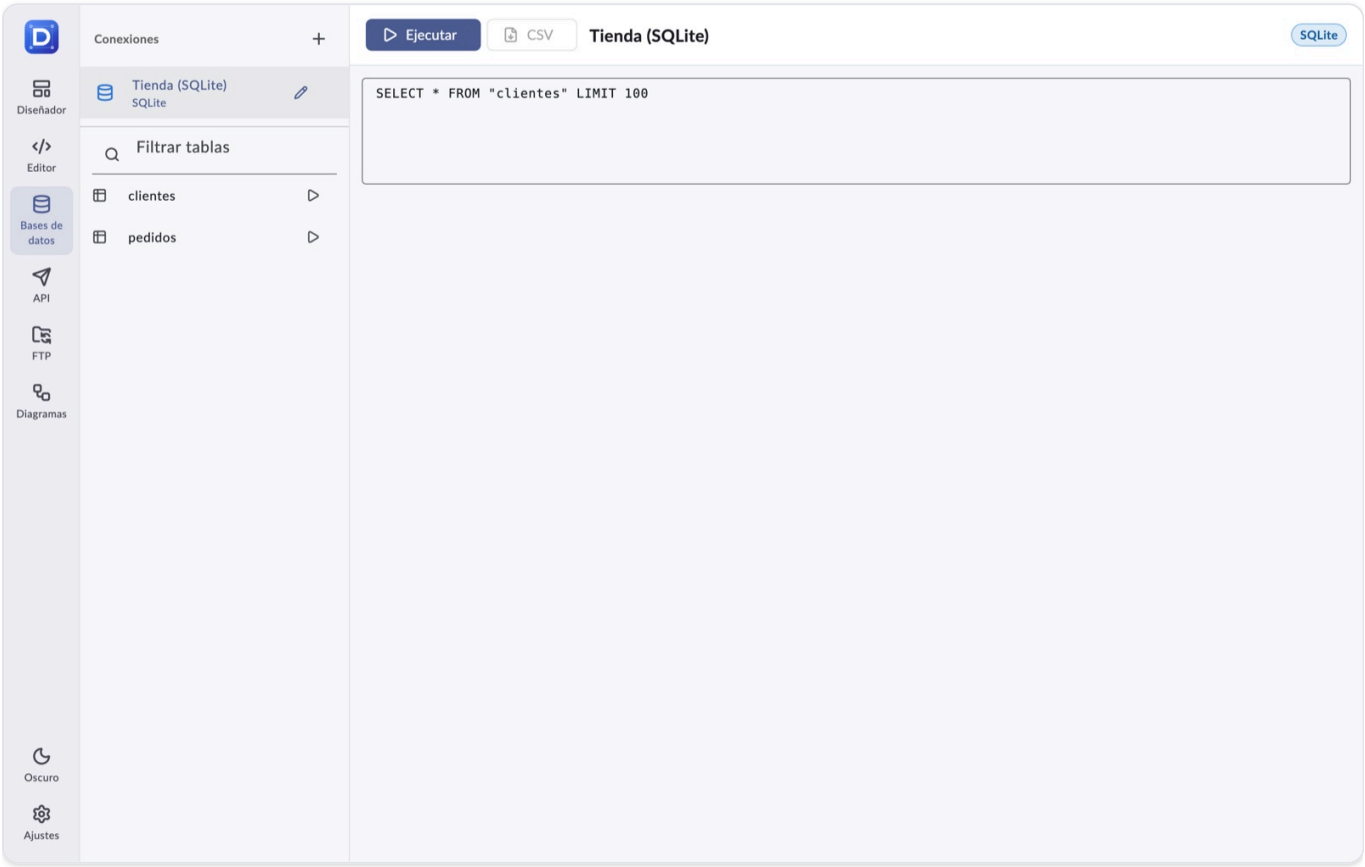
Bases de datos

Conexiones, tablas, consultas SQL y exportación a CSV.

Conexiones

Con **+** se añade una conexión de tipo SQLite, PostgreSQL, MySQL / MariaDB, MongoDB, FalconDatabase o Firebase, con servidor, puerto, base de datos, usuario, contraseña y **Conexión cifrada (SSL)** según el tipo. **Probar conexión** comprueba que se llega.

Las conexiones guardadas aquí son las que pueden usar las ventanas del Diseñador con el componente **Conexión de datos**.



Una base SQLite con sus tablas y el editor de consultas.

Tablas y consultas

- La lista de tablas se puede **Filtrar tablas**; cada tabla abre una consulta de ejemplo.
- **Ejecutar** (⌘↵) lanza la consulta del editor y muestra el resultado en una rejilla.
- **Scripts**: con varias consultas en el editor (separadas por `;` en SQL, o un JSON tras otro en MongoDB, FalconDatabase y Firebase) se ejecutan todas en orden y cada una deja su resultado en una pestaña (**1**

· 5 filas, **2 · Hecho**…). Si una falla, se para ahí y se ve el error con los resultados de las anteriores.

- Si hay texto **seleccionado**, solo se ejecuta la selección.
- En los scripts SQL no cuentan los `;` de las cadenas, los comentarios, los bloques `$$ … $$` de PostgreSQL ni el cuerpo `BEGIN … END` de un disparador de SQLite; en MySQL, `DELIMITER //` cambia el separador para los procedimientos, como en MySQL Workbench. En JSON se admiten comentarios `//`.
- Antes de ejecutar algo que cambia datos se pide confirmación (una vez por script).
- **CSV** guarda el resultado completo en un archivo.

FalconDatabase y Firebase

También se conecta con bases de datos en la nube por su API: **FalconDatabase** (tablas de PostgreSQL) y **Firebase** (Cloud Firestore). Sus consultas son un JSON, como en MongoDB.

Tipo	Qué pide	Consulta de ejemplo
FalconDatabase	Servidor (por defecto falcondatabase.com), la referencia del proyecto y su clave (`fdb_anon_…` o `fdb_service_…`); correo y contraseña si las tablas piden sesión	`{"table": "productos", "where": {"precio": "gte.10"}, "order": "id.desc", "limit": 100}`
Firebase	El ID del proyecto, la clave web (`apiKey`) y, si las reglas piden sesión, un usuario de Firebase Authentication; o un emulador local	`{"collection": "productos", "where": {"precio": {"gte": 10}}, "order": "precio desc"}`

- Para cambiar datos: `"insert": {…}` o una lista, `"update": {"id": …, "set": {…}}` y `"delete": id`. En los dos, `update` y `delete` también aceptan `"where": {…}` en lugar del id: cambian o borran todas las filas que cumplen el filtro. Se pide confirmación antes.
- FalconDatabase: `"select": "id,nombre"`, `"order"`, `"limit"` y `"offset"`; al listar dice cuántas filas hay en total.
- Firebase: subcolecciones como ruta (`"collection": "pedidos/p1/lineas"`), `"group": true` para buscar en todas las subcolecciones con ese nombre, `"id"` para un documento, `"select"`, `"offset"`, orden por varios campos (`"precio desc, nombre"`) y los agregados `"count": true`, `"sum"` y `"avg"`.
- MongoDB: además de `find` (con `projection`, `sort`, `limit` y `skip`), `aggregate`, `insert`, `update` y `delete`, también `"count": true`, `"distinct": "campo"`, `"createIndex": {…}` (con `"unique"`), `"indexes": true` y `"drop": true`.
- En FalconDatabase los filtros usan sus operadores: `eq`, `neq`, `gt`, `gte`, `lt`, `lte`, `like`, `ilike`, `is` e `in`. En Firebase: `eq`, `neq`, `gt`, `gte`, `lt`, `lte`, `in` y `contains`.

- En Firebase cada fila lleva `_id`, el identificador del documento. Los operadores también incluyen `nin` y `containsAny`.
- En la web hay un curso de cada base de datos (SQLite, PostgreSQL, MySQL, MongoDB, FalconDatabase y Firebase), con scripts que se ejecutan aquí tal cual: **Cursos** en el menú.
- Por ahora son para el módulo Bases de datos: el componente **Conexión de datos** del Diseñador usa SQLite, PostgreSQL y MySQL.

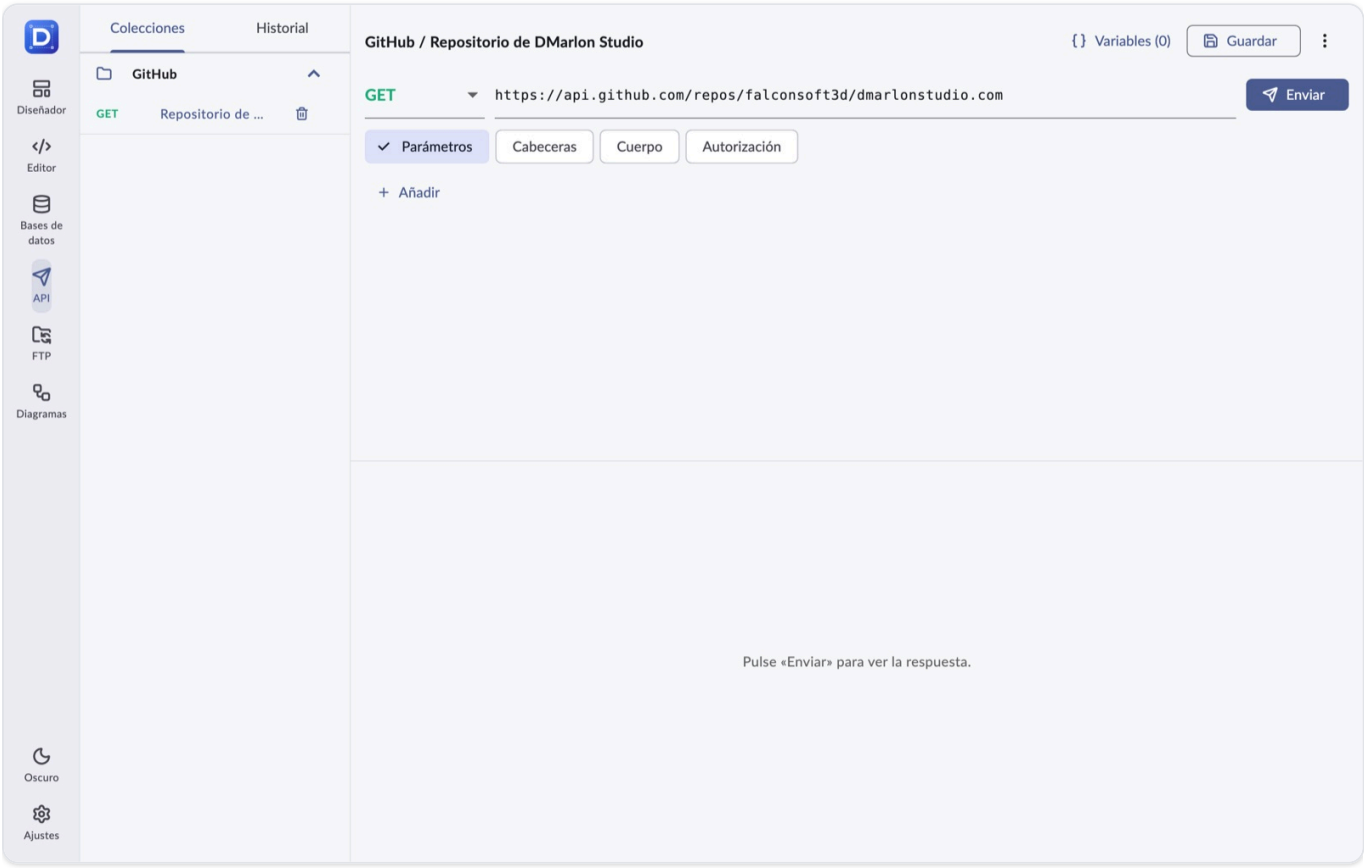
Capítulo 16

Cliente de API

Peticiones HTTP con colecciones, variables e historial.

Peticiones

- Elija el método (GET, POST, PUT, PATCH, DELETE…), escriba la URL y pulse **Enviar**.
- Pestañas **Parámetros**, **Cabeceras**, **Cuerpo** (JSON, formulario o texto) y **Autorización** (ninguna, Bearer o Basic).
- La respuesta muestra el código, el tiempo, las cabeceras y el cuerpo.
- **Copiar como curl** deja la petición lista para la terminal.



Una petición guardada en la colección GitHub.

Colecciones, variables e historial

- **Guardar** guarda la petición en una **Colección**; **Guardar como nueva…** hace una copia.
- **Variables de entorno…** define valores como `base_url` o `token`, que se usan en la URL, las cabeceras y el cuerpo como `{{base_url}}`.

- El **Historial** guarda las 100 últimas peticiones enviadas; **Borrar historial** lo vacía.

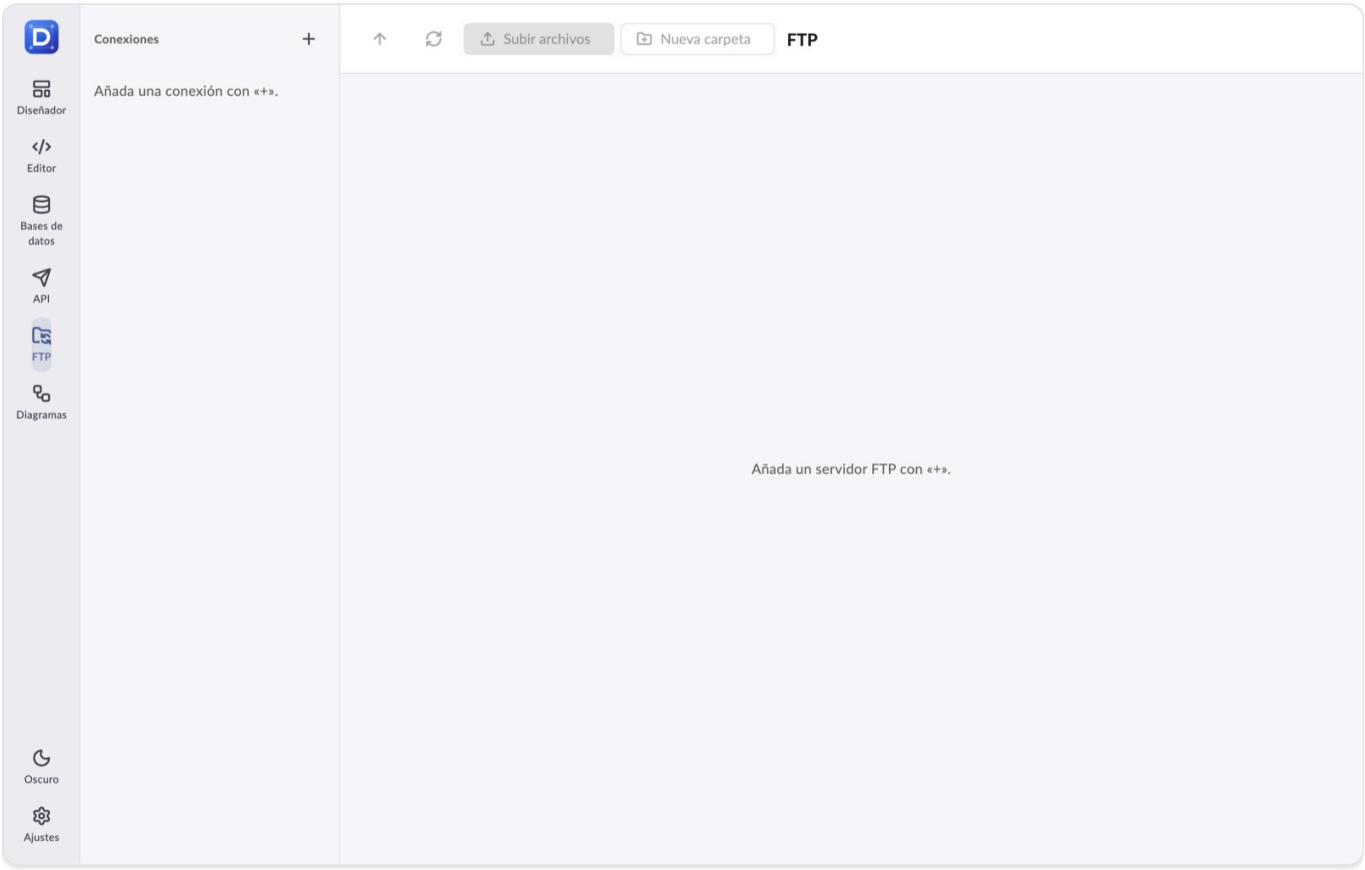
Capítulo 17

FTP

Servidores FTP, FTPES y FTPS.

Servidores y archivos

- Con **+** se añade un servidor con su **Seguridad**: **FTP (sin cifrar)**, **FTPES (TLS explícito)** o **FTPS (TLS implícito)**.
- Elija un servidor para recorrer sus carpetas; **Carpeta superior** sube un nivel.
- **Subir archivos**, **Descargar**, **Nueva carpeta**, **Renombrar…** y **Eliminar** (las carpetas, con su contenido).



El cliente FTP, listo para añadir un servidor.

Capítulo 18

Diagramas

Diagramas de flujo y de bases de datos, con IA y PDF.

Diagramas de flujo y de base de datos

- **De flujo**: figuras **Inicio / fin**, **Proceso**, **Decisión**, **Entrada / salida** y **Nota**. Pulse → junto a una figura para conectarla con otra; las conexiones pueden llevar etiqueta (p. ej. «Sí»).
- **De base de datos**: tablas con sus columnas, **Clave primaria** y claves ajenas, y las relaciones entre tablas. **SQL del diagrama** muestra el SQL que lo crea.
- **Desde una base de datos** dibuja el diagrama de una conexión guardada (opcionalmente, solo las tablas que empiezan por un prefijo).
- **Con IA** crea el diagrama a partir de una descripción y **Cambiar con IA** lo modifica.
- **Ordenar** recoloca las figuras y **Exportar a PDF** lo guarda para imprimir o compartir.



Un diagrama de flujo para dar de alta un cliente.

Capítulo 19

Ajustes

Idioma, apariencia e inteligencia artificial.

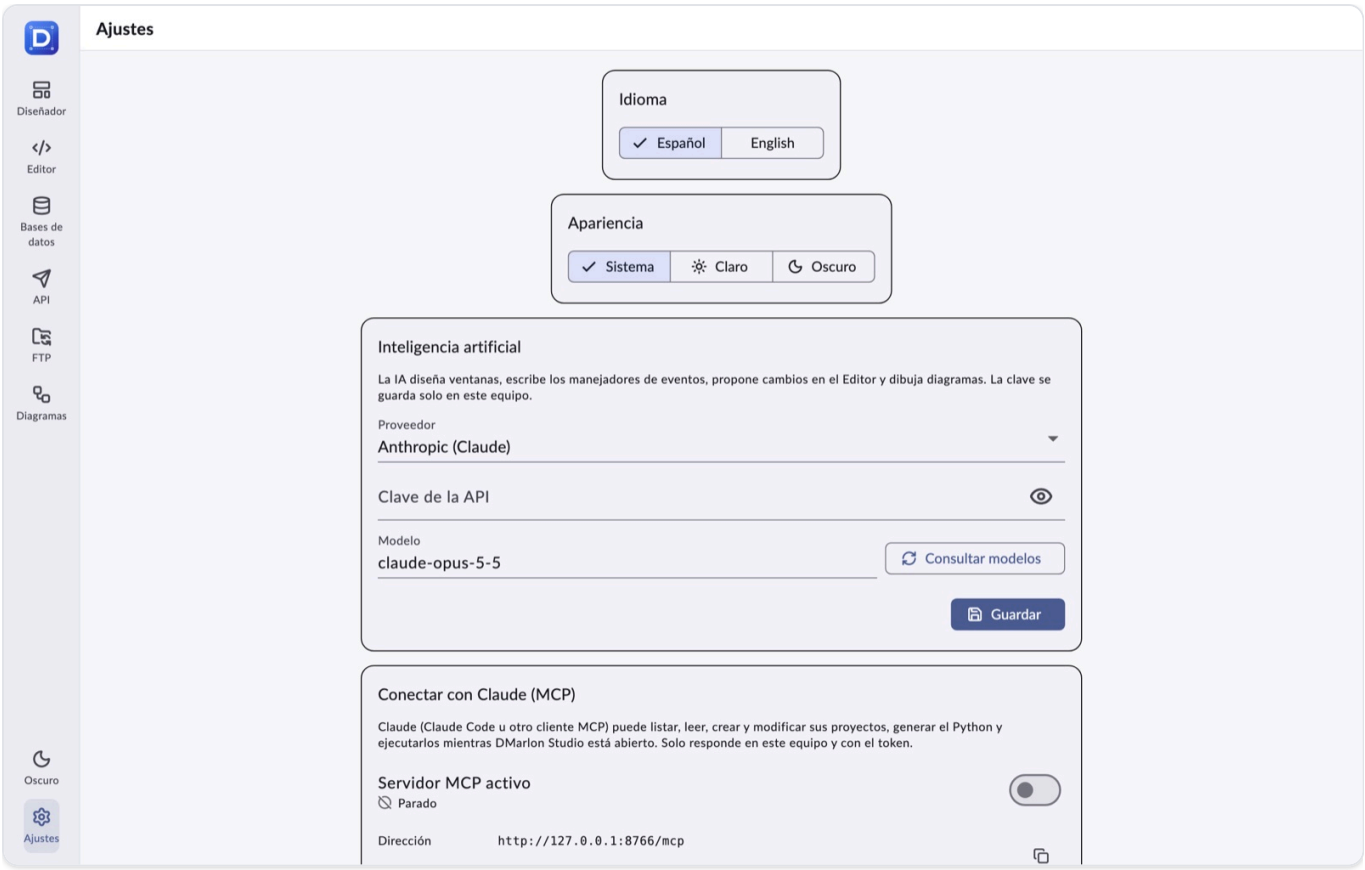
Conectar con Claude (MCP)

- En **Ajustes → Conectar con Claude (MCP)** se activa el **Servidor MCP**: mientras DMarlon Studio está abierto, Claude (Claude Code u otro cliente MCP) puede trabajar con sus proyectos.
- Herramientas: `list_projects`, `get_project`, `list_components`, `create_project`, `add_component`, `update_component`, `remove_component`, `update_code`, `generate_python`, `run_project`, `get_output` y `stop_project`.
- Solo responde en este equipo (`127.0.0.1`) y con el **Token**. **Añadir a Claude Code** lo da de alta con un clic; también se puede copiar la orden `claude mcp add` que se muestra.
- Si un proyecto está abierto en el Diseñador sin cambios pendientes, se recarga solo con lo que haga Claude; con cambios sin guardar, Claude recibe un aviso para que primero se guarden.

Inteligencia artificial

Elija el **Proveedor**: Anthropic (Claude), OpenAI, Gemini, xAI, DeepSeek, Mistral, Groq, Cohere, OpenRouter, Perplexity, Ollama o una API personalizada compatible. Escriba la **Clave de la API** y el **Modelo**; **Consultar modelos** trae los de su cuenta. Claude (`claude-opus-5-5`) es el proveedor por defecto.

La clave se guarda solo en este equipo. Cada proveedor recuerda su clave y su modelo al cambiar de uno a otro.



Ajustes: idioma, apariencia, inteligencia artificial y versión.

Idioma, apariencia y versión

- **Idioma**: Español o English, al momento.
- **Apariencia**: Sistema, Claro u Oscuro.
- **Acerca de** muestra la versión instalada.

Capítulo 20

Atajos de teclado

Todos los atajos del Diseñador y del Editor.

Diseñador

Atajo	Acción
F9	Ejecutar (depurando y detenido: continuar)
Mayús+F9	Depurar
F5	Poner o quitar un punto de parada en la línea del cursor
F7 / F8 / Mayús+F8	Entrar / pasar por encima / salir de la función
F4	Ejecutar hasta el cursor
Ctrl+F2	Detener
F12	Pasar del diseño al código y vuelta
Ctrl+Espacio	Autocompletar en el código
F1 / clic derecho → Ayuda	Ayuda de la palabra del cursor
⌘F / Ctrl+F	Buscar (y reemplazar) en el código
⌘S / Ctrl+S	Guardar el proyecto
⌘Z / Ctrl+Z	Deshacer
⌘C, ⌘X, ⌘V (Ctrl en Windows)	Copiar, cortar y pegar componentes
Supr / Retroceso	Eliminar la selección
Flechas	Mover un paso de rejilla (con Alt, un píxel)
Mayús + flechas	Cambiar el tamaño
Doble clic en la paleta	Añadir el componente
Doble clic en un componente	Abrir su evento principal
Intro en un chat	Enviar (Mayús+Intro: salto de línea)

Editor

Atajo	Acción
⌘S / Ctrl+S	Guardar el archivo
⌘J / Ctrl+J	Mostrar u ocultar la terminal
↑ / ↓ en la terminal	Recorrer el historial de órdenes
Intro en un chat	Enviar (Mayús+Intro: salto de línea)

Capítulo 21

Solución de problemas

Los avisos más habituales y cómo resolverlos.

No encuentra Python o le falta tkinter

- **No se encontró Python**: instálelo (python.org o Homebrew) y pulse **Volver a buscar Python** en la lista de proyectos.
- **Se encontró … pero no trae tkinter**: en macOS, `brew install python-tk`; en Linux, `sudo apt install python3-tk`; en Windows, reinstale Python marcando **tcl/tk and IDLE**.
- **Tk 8.5 es antiguo y no respeta los colores del diseño**: es el Python que trae macOS. Instale uno de python.org o Homebrew.

Falta Flask

Las aplicaciones web necesitan Flask en el mismo Python. El aviso **Falta Flask** muestra la orden exacta, por ejemplo `python3 -m pip install flask`.

La app no abre (sin firmar)

- **macOS**: la primera vez, clic derecho sobre DMarlon Studio → **Abrir** → **Abrir**.
- **Windows**: en «Windows protegió su PC», **Más información** → **Ejecutar de todas formas**.

AttributeError: 'Form1' object has no attribute 'root'

Pasaba cuando el código de la ventana definía su propio `__init__` sin llamar a `super().__init__()`: ese método sustituía al que crea la ventana. Desde la versión 1.0.5 la ventana y los componentes se crean antes de su `__init__`, y el evento **Al abrir** se ejecuta después.

```
class Form1(FormDesign):
    def __init__(self):
        super().__init__()   # primero
        self.current = "0"

    # Mejor aún: el evento Al abrir
    def form_create(self):
        self.current = "0"
```

No se encontró Claude Code

Instale Claude Code desde claude.com/claude-code y ejecute `claude` una vez en una terminal para iniciar sesión. El estudio lo busca en el PATH y en `~/.local/bin`.

Capítulo 22

Hoja de ruta

Lo que viene después.

Destinos

Al crear un proyecto se elige el destino; no se puede cambiar después:

Destino	Estado
Python + tkinter (escritorio)	Disponible
Python + Flask (web)	Disponible

Proyectos más completos

- Varios formularios por proyecto, con un nodo **Proyecto** en el árbol.
- El icono de la aplicación, definido en el proyecto.
- Compilar ejecutables (PyInstaller para Python).